STM kretačka

Dokumentacija koda — svaki fajl, svaka funkcija, red po red

Projekat	Eurobot 2027 — pogonska ploča robota
Mikrokontroler	STM32H750VBTx
Uloga ploče	reguliše brzinu levog i desnog točka u rad/s, javlja stanje preko CAN-a
Nadređeni sistem	Raspberry Pi (ros2 control), komunikacija preko CAN 500 kbps
Putanja projekta	`D:\Desktop\Eurobot2027\kod 1.18`
Verzija	rad/s, pozicioni PID sa D članom i filterima, odometrija isključena (radi je ros2)

> Ovaj dokument opisuje **samo kod koji je pisan za ovu ploču** — folder `Core/Src/Lib` i `main.c`. Fajlovi koje pravi CubeMX (`tim.c, gpio.c, fdcan.c, adc.c...`) i tvoji raniji fajlovi (`motor.c, motor2.c, ADC.c, comm.c`) nisu menjani i pominju se samo tamo gde je bitno šta od njih zavisi. Svi listinzi koda u dokumentu su **izvučeni direktno iz fajlova projekta**, sa pravim brojevima linija.

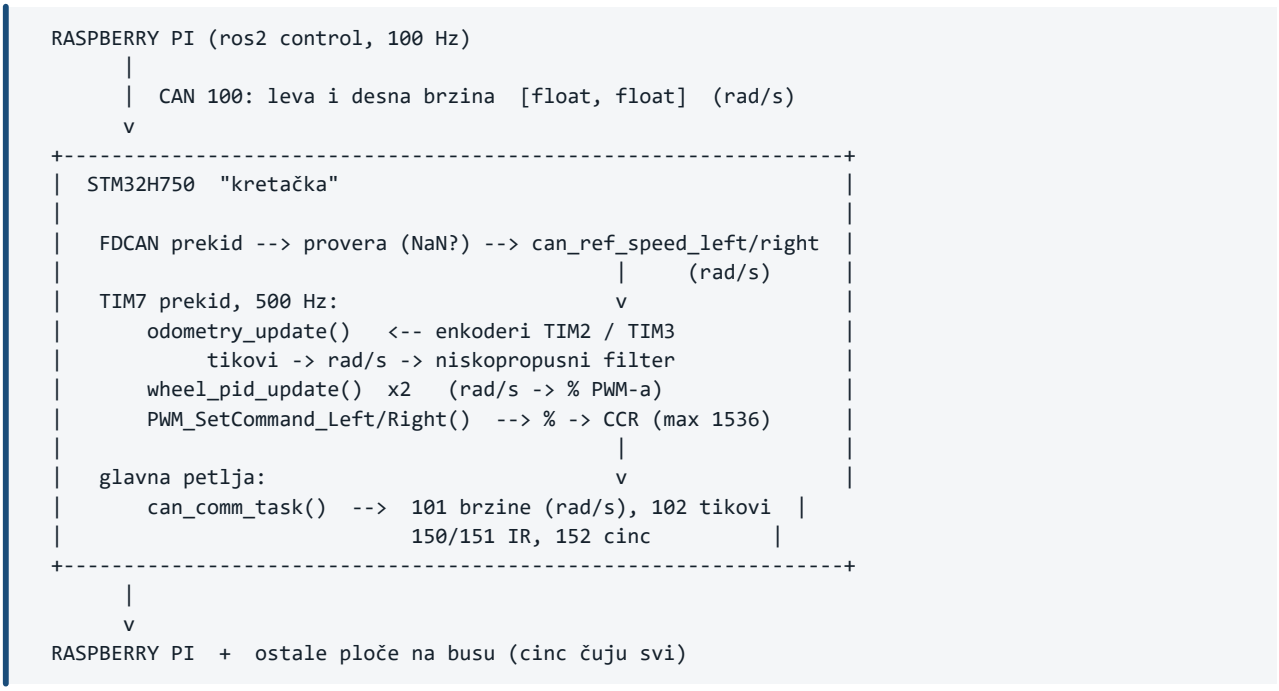
1. Šta ova ploča radi

Ceo posao je podeljen na dva računara. Raspberry Pi računa **šta** robot treba da uradi, a STM se bavi time **kako** se to izvede na motorima.

Raspberry Pi (ros2)	STM kretačka (ovaj kod)
zna gde je robot na terenu i kuda ide	ne zna ništa o terenu, strategiji ni poziciji
računa odometriju (x, y, ugao) iz enkodera koje mu STM šalje	ne računa poziciju — samo broji tikove i meri brzinu točka
računa putanju, rotaciju, krivolinijsko kretanje	samo drži zadatu brzinu svakog točka
šalje željenu brzinu svakog točka u **rad/s**, 100 puta u sekundi	meri stvarnu brzinu u **rad/s** i podešava PWM 500 puta u sekundi
prima nazad izmerene brzine (rad/s) i brojače enkodera	šalje ih 100 puta u sekundi
prima stanje IR senzora i cinca	čita te pinove i javlja kad se promene

Posledica ove podele: **u kodu STM-a nema milimetara**. Sve brzine su ugaone brzine točka u rad/s, i to od CAN poruke, kroz regulator, do feedback-a. Prečnik točka i razmak točkova STM-u nisu potrebni — to su podaci koje koristi ros2 kad iz tikova računa gde je robot. Zato ne postoji ni kalibracija prečnika na ovoj strani: ako ros2 zada 10 rad/s, točak se okreće 10 rad/s, tačka.

1.1 Tok podataka



1.2 Dve petlje koje treba razlikovati

Ceo program se vrti u dve nezavisne petlje. Ovo je najvažnija stvar za razumevanje koda — ako pomešaš šta se gde dešava, ništa neće imati smisla.

	TIM7 prekid	glavna petlja `while(1)`
koliko često	tačno 500 puta u sekundi (svake 2 ms)	neprekidno, koliko god brzo procesor stigne

	TIM7 prekid	glavna petlja `while(1)`
šta radi	enkoderi → brzina → PID → PWM	slanje CAN poruka, čitanje IR i cinca, provera timeout-a
zašto baš tu	regulacija mora da bude na tačnom, ravnomernom taktu	slanje poruka sme da kasni koji milisekund, nije kritično
gde je u kodu	`HAL_TIM_PeriodElapsedCallback()` u `main.c`	`can_comm_task()` u `can_comm.c`

> Treći „tok" je **FDCAN prekid** — on se javi kad stigne poruka sa CAN-a, proveri da li je broj ispravan, upiše ga u promenljivu i odmah se vrati. Ne računa ništa. Sve promenljive koje on deli sa TIM7 prekidom su `volatile`.

1.3 Spisak fajlova

fajl	šta je unutra
`board_config.h`	sva podešavanja na jednom mestu: frekvencije, granice napona, enkoder, filteri, pinovi senzora, početna pojačanja. **Ovde se menjaju brojevi.**
`lowpass.h`	niskopropusni filter prvog reda — dve male inline funkcije. Koriste ga PID (D član) i odometrija (brzina).
`PID.h / PID.c`	pozicioni PID regulator brzine sa anti-windup-om i filtriranim D članom. Jedan primerak po točku, **zajednički koeficijenti**.
`encoders.h / encoders.c`	čitanje hardverskih brojača enkodera (TIM2 i TIM3).
`odometry.h / odometry.c`	tikovi → brzina točka u rad/s (sirova i filtrirana) + ukupni tikovi. Kod za x, y i ugao je unutra ali **isključen** (`ODOMETRY_ON_STM`).
`motors.h / motors.c`	komanda u % → smer + ispuna; plafon ispune iz napona napajanja; ovde je zaštita od 18 V.
`can_comm.h / can_comm.c`	ceo CAN protokol: prijem komandi (sa proverom), slanje povratnih informacija, IR senzori, cinc.
`main.c`	pokretanje svega redom i regulaciona petlja u TIM7 prekidu.

Fajlovi `motor.c` i `motor2.c` su tvoji raniji fajlovi — nisu menjani. `motors.c` (množina) je sloj iznad njih.

2. Šta se dešava od paljenja ploče

Redosled je bitan i nije slučajan. Evo šta se izvršava, redom:

#	korak	zašto baš tu
1	`MPU_Config()`, `HAL_Init()`, `SystemClock_Config()`	standardni CubeMX start — uključuje takt procesora i periferija
2	svi `MX_*_Init()` (GPIO, DMA, ADC, TIM1, I2C, UART, TIM2, TIM3, TIM7, FDCAN1)	CubeMX podešava periferije; ništa još ne radi
3	ručno nameštanje TIM7 na 500 Hz	da frekvencija regulacije ne zavisi od toga šta je u CubeMX-u
4	`wheel_pid_gains_init()` pa `wheel_pid_init()` za levi i desni	prvo se napravi **jedan** skup koeficijenata, pa se oba regulatora povežu na njega
5	`PWM_Init()`	računa plafon ispune iz napona, budi oba drajvera, PWM na 0 %
6	`encoder_init()`, `odometry_init()`	startuje brojače enkodera, računa koeficijent filtera brzine, nulira zbirove
7	`can_comm_init()`	filter, start FDCAN-a — ali **bez** prekida
8	`can_comm_startup_flush()`	prazni primljeno smeće i šalje 2000 nula; tek na kraju uključuje RX prekid
9	`HAL_TIM_Base_Start_IT(&htim7)`	**od ovog trenutka regulator radi** — namerno je poslednji
10	`while(1) { can_comm_task(); }`	dalje sve ide kroz prekide

2.1 Zašto se prekid uključuje tek na kraju

Dve stvari se namerno odlažu:

RX prekid se uključuje tek posle flush-a. Ako je robot pre toga bio ugašen nasred vožnje, u Linux baferu na Raspberry strani može da stoji stara, neizvršena komanda „vozi 15 rad/s". Kad se STM upali, ta poruka bi stigla prva i robot bi naglo krenuo. Zato se prvo sve primljeno baci, pa se tek onda počne slušati.

TIM7 se startuje poslednji. Da regulator ne bi počeo da radi dok enkoderi i motori još nisu inicijalizovani.

> Ovo nije teorija — ista stvar se dešavala na Memristoru, i zato je u planu protokola i stajalo „u main pre glavne petlje na STM pošalji par hiljada nula".

2.2 Gde će doći ADC

Napon napajanja motora je **promenljiva** `motor_supply_voltage`, ne makro. Za sada ima početnu vrednost 30 V i niko je ne menja, pa je plafon ispune fiksnih 60 %. Kad se doda merenje baterije, u `main()` pre `PWM_Init()` ide jedna linija:

```
PWM_SetSupplyVoltage(ADC_ReadVbat());   /* pre PWM_Init() */
```

i plafon se sam preračuna tako da motor i dalje dobija tačno 18 V pri punoj komandi. Ništa drugo u kodu ne mora da se dira.

3. board_config.h — sva podešavanja na jednom mestu

U ovom fajlu nema nijedne funkcije, samo `#define`-ovi. Ideja je da kad nešto treba da se promeni (frekvencija, granica napona, filter, pojačanja), menjaš **samo ovde**, a ne da tražiš broj rasut po pet fajlova.

3.1 Kontrolna petlja

kod	šta radi
`#define CONTROL_FREQ_HZ 500`	Koliko puta u sekundi radi regulator. Koristi se za ARR registar TIM7, za pretvaranje tikova u rad/s, za koeficijente oba filtera i za deljenje na 100 Hz feedback. Ako ovo promeniš, sve se samo prilagodi.
`#define CONTROL_PERIOD_S \ (1.0f / (float)CONTROL_FREQ_HZ)`	Perioda T = 0.002 s. Ovo je ono T iz formule za filter, `a = e^(-2·π·fc·T)`.
`#define FEEDBACK_FREQ_HZ 100`	Koliko često se šalju poruke 101 i 102 nazad. 500/100 = svaki peti prolaz kroz prekid. Ne šalje se 500 puta da se ne preoptereti procesor na Raspberry strani.
`#define TIM7_INPUT_CLK_HZ 64000000UL`	Takt koji ulazi u TIM7 (APB1 Timer Clock iz CubeMX-a). **Ako promeniš SYSCLK, moraš i ovaj broj** — inače regulator radi na pogrešnoj frekvenciji, a to se teško primeti.
`#define TIM7_PRESCALER 63`	Deli 64 MHz sa 64 i dobija se brojački takt od 1 MHz (jedan tik = 1 µs).

3.2 Motori i zaštita napona

Regulator ne zna ništa o voltima ni o CCR registru — njegov izlaz je **procenat maksimalnog dozvoljenog PWM-a**, od −100 do +100. Šta je „maksimalni dozvoljeni" određuje se ovde:

kod	šta radi
`#define MOTOR_SUPPLY_VOLTAGE_DEFAULT 30.0f`	**Početna** vrednost napona napajanja. Sam napon je promenljiva `motor_supply_voltage` u `motors.c`, da bi mogla da se učita sa ADC-a na startu. Dok se to ne uradi, važi ovih 30 V.
`#define MOTOR_MAX_VOLTAGE 18.0f`	Najviše što motor sme da dobije. Iz ova dva broja motors.c računa plafon ispune: 18 / 30 × 100 = **60 %** = CCR 1536 od 2560. Komanda +100 % = tačno taj CCR.
`#define MOTOR_SUPPLY_VOLTAGE_MIN 12.0f` `#define MOTOR_SUPPLY_VOLTAGE_MAX 40.0f`	Razumne granice za izmereni napon. Ako ADC jednog dana vrati 0 V ili 200 V (otkačen kabl, pogrešan delitelj), vrednost se odbija i ostaje prethodna — jer bi 0 V dalo plafon od 100 % i punih 30 V na motoru.

3.3 Enkoderi

kod	šta radi
`#define ENC_LEFT_TIMER  htim2` `#define ENC_RIGHT_TIMER htim3`	Koji tajmer broji koji enkoder. TIM2 = ENC1 (PA15/PB3), 32-bitni. TIM3 = ENC2 (PC6/PC7), 16-bitni. Ako zamenite konektore, dovoljno je zameniti ova dva reda.
`#define ENCODER_TICKS_PER_REV \` `  (2048 * 4)`	Koliko tikova brojač izbroji za jedan pun obrtaj točka. Enkoder ima 2048 impulsa, a tajmeri su u modu `TIM_ENCODERMODE_TI12` koji broji **obe ivice na oba kanala** — otud ×4, ukupno 8192. **Ovo je jedini broj koji povezuje enkoder sa fizikom** — nema prečnika, nema obima.
`#define ENC_LEFT_SIGN  (+1)` `#define ENC_RIGHT_SIGN (+1)`	Ako neki enkoder broji unazad kad točak ide napred (zamenjeni A i B kanal), staviš −1 i sve se obrne na jednom mestu, bez prelemljivanja.

`#define RAD_PER_TICK \`
`  (6.283185307f / \`
`  (float)ENCODER_TICKS_PER_REV)`
	2π podeljeno sa 8192 = **0.000767 rad po tiku**. Jedini faktor koji pretvara tikove u ugao. Brzina je onda: tikovi po prolazu × ovo × 500.

3.4 Filter izmerene brzine

kod	šta radi
`#define SPEED_LPF_FC_HZ 30.0f`	Granična frekvencija filtera na izmerenoj brzini. Brzina iz jednog prolaza od 2 ms je kvantizovana na cele tikove: jedan tik više ili manje je ±0.38 rad/s. Filter to umiri **pre nego što uđe u PID** — a ista filtrirana brzina se šalje i na CAN. Niže = mirnije ali kasni više; 30 Hz je kašnjenje od ~5 ms, zanemarljivo.
`#define MAX_WHEEL_SPEED_RADS 60.0f`	Zadata brzina sa CAN-a se odseca na ±60 rad/s. Fizički maksimum je ~29 rad/s, pa je ovo čisto zaštita od gluposti (pogrešan bajt, pogrešna jedinica na ROS strani).

3.5 Digitalni ulazi — IR senzori i cinc

kod	šta radi
`#define IR_ACTIVE_HIGH 1`	1 znači da **visok napon na pinu = ima prepreke**. Ako su ti senzori obrnuti, staviš 0 i ceo kod se prilagodi.
`#define IR_DEBOUNCE_MS 5`	Koliko dugo ulaz mora da bude miran pre nego što se promena prizna. Kratko, jer prepreka mora brzo da se javi.
`#define CINC_Pin GPIO_PIN_6` `#define CINC_GPIO_Port GPIOE`	Pin na koji je zakačen cinc. **Ovo je privremena vrednost** — ako pinu u CubeMX-u daš labelu `CINC`, CubeMX sam napravi ove definicije u `main.h`, pa ove ovde treba obrisati.
`#define CINC_ACTIVE_HIGH 1`	1 znači: **visok napon = cinc je izvučen = start meča**. Podrazumeva ožičenje gde kabl kratko spaja pin na masu dok je utaknut (pin je PULL-UP).
`#define CINC_DEBOUNCE_MS 20`	Duže nego za IR, jer je cinc mehanički kontakt i pri izvlačenju „poskakuje". 20 ms pouzdano odbacuje to poskakivanje, a čoveku je i dalje trenutno.

Kod ovakvog ožičenja, **prekinuta žica ili ispao konektor se čita kao „izvučen" i robot kreće**. Obrnuto ožičenje ima istu manu, samo se zamene uloge — sa jednim pinom nema varijante bez te opasnosti. Proveri kontakt pre meča.

3.6 Regulator — početna pojačanja i granice

kod	šta radi
`#define PID_DEFAULT_KP 5.7f`	Proporcionalno pojačanje, u **% po rad/s greške**. Menja se porukom 103.
`#define PID_DEFAULT_KI 0.118f`	Integralno pojačanje, **po prolazu** (T = 2 ms je već uračunato, kao i u PIC32 kodu). Menja se porukom 104.
`#define PID_DEFAULT_KD 0.0f`	Izvodno pojačanje. Nula — D član postoji u kodu i radi, ali ros2 šalje 0 pa je isključen.
`#define PID_D_LPF_FC_HZ 20.0f`	Granična frekvencija filtera na D članu. Izvod pojačava šum, zato se filtrira; 20 Hz daje koeficijent a = 0.778.
`#define PID_COMMAND_LIMIT_PCT 100.0f`	Izlaz regulatora se klešti na ±100 %. Preko toga nema smisla — 100 % je već maksimalni CCR.
`#define PID_INTEGRATOR_LIMIT_PCT 100.0f`	Sam I član se klešti na ±100 %. Pri stalnoj brzini P član pada na nulu i **ceo** izlaz nosi integrator, pa mora da može da dođe do 100 %. (U PIC32 kodu je bilo pola opsega — to bi ovde ostavilo trajnu grešku pri velikim brzinama.)

Odakle 5.7 i 0.118

Stari regulator je radio u mm/s sa Kp = 2.8 i Ki = 0.058, a izlaz mu je bio „brzina" od 0 do 2000 koja se skalirala na 60 % PWM-a. Novi radi u rad/s sa izlazom u %. Da odziv ostane **identičan**, pojačanja se preračunaju: greška u mm/s je greška u rad/s × 40.77 (poluprečnik), a izlaz od 2000 mm/s je 100 %, dakle deljenje sa 20:

```
Kp = 2.8   × 40.77 / 20 = 5.71
Ki = 0.058 × 40.77 / 20 = 0.118
```

Provereno simulacijom na istom modelu motora: vreme do 90 % zadate brzine 160 ms (staro 166 ms), preskok 0.06 % (staro 0.11 %), ustaljena vrednost tačna u oba slučaja.

3.7 Odometrija — isključena

```
#ifndef ODOMETRY_ON_STM
  #define ODOMETRY_ON_STM 0
#endif
#if ODOMETRY_ON_STM
  #define WHEEL_DIAMETER_MM     81.54f
  #define WHEEL_RADIUS_MM       (WHEEL_DIAMETER_MM / 2.0f)
  #define WHEEL_BASE_MM         422.0f
  #define RADS_TO_MMS(w)  ((w) * WHEEL_RADIUS_MM)
  #define MMS_TO_RADS(v)  ((v) / WHEEL_RADIUS_MM)
#endif
```

Dimenzije robota postoje samo iza ovog prekidača, jer ih koristi jedino računanje pozicije — a to radi ros2. Kod za x, y i ugao je ostao u `odometry.c`, isto iza `#if ODOMETRY_ON_STM`, i **ne kompajlira se**. Ako ikad zatreba na STM-u: staviš 1, proveriš dimenzije, i sve se vrati. Proverio sam da se kompajlira u obe varijante.

4. lowpass.h — niskopropusni filter prvog reda

Najosnovniji filter koji postoji: nova vrednost je mešavina stare filtrirane i nove sirove. Koristi se na dva mesta — na izmerenoj brzini točka i na D članu regulatora — pa je izdvojen u zaseban mali fajl da se ista formula ne piše dvaput.

Core/Src/Lib/lowpass.h

```
18  #ifndef LIB_LOWPASS_H_
19  #define LIB_LOWPASS_H_
20  #include <math.h>
21
22  static inline float lowpass_coefficient(float fc_hz, float sample_freq_hz)
23  {
24      if (fc_hz <= 0.0f || sample_freq_hz <= 0.0f)
25          return 0.0f;                                    /* bez filtriranja */
26      return expf(-6.283185307f * fc_hz / sample_freq_hz);
27  }
28
29  static inline float lowpass_step(float coefficient, float filtered_old, float raw)
30  {
31      return coefficient * filtered_old + (1.0f - coefficient) * raw;
32  }
33
34  #endif /* LIB_LOWPASS_H_ */
```

kod	šta radi
`filtrirano = a * filtrirano_staro + (1 - a) * sirovo`	Formula filtera. `a` je između 0 i 1: koliko „pamćenja" ima filter. a = 0 → nema filtriranja (prolazi sirovo). a → 1 → jako sporo.
`a = e^(-2 * pi * fc * T)`	Koeficijent iz granične frekvencije fc i periode odabiranja T. **Minus u eksponentu je obavezan** — bez njega je a > 1 i filter divergira (svaki prolaz uveća vrednost).
`static inline float lowpass_coefficient(float fc_hz, float sample_freq_hz)`	Računa a. Zove se jednom, na startu, za svaki filter posebno (fc su različiti). `expf` je float verzija eksponencijalne funkcije iz `math.h` — na H7 sa FPU to je nekoliko mikrosekundi, nebitno na startu.
`if (fc_hz <= 0.0f || sample_freq_hz <= 0.0f) return 0.0f;`	Zaštita: fc = 0 ili negativan takt bi dali besmislen a. Vraća 0 = filter isključen.
`static inline float lowpass_step(float coefficient, float filtered_old, float raw)`	Jedan korak filtera. `static inline` znači da kompajler ubaci ovo direktno na mesto poziva — nema troška poziva funkcije u prekidu.

fc	T = 2 ms (500 Hz)	a	kašnjenje
10 Hz		0.882	~16 ms
20 Hz (D član)		**0.778**	~8 ms
30 Hz (brzina)		**0.686**	~5 ms
50 Hz		0.533	~3 ms

5. PID.h / PID.c — regulator brzine točka

Pozicioni PID po uzoru na tvoj PIC32 kod (`pid_regulator.c`): proporcionalni + integralni sa anti-windup-om + izvodni kroz niskopropusni filter. Postoje dva primerka, po jedan za svaki točak, koji dele **jedan skup koeficijenata**.

```
error       = reference - feedback                        [rad/s]
P           = Kp * error                                   [%]
I          += Ki * error        (klešten, anti-windup)
D_raw       = error - error_previous                      (po prolazu)
D_filtered  = a * D_filtered + (1 - a) * D_raw,   a = e^(-2·π·fc·T)
command     = P + I + Kd * D_filtered   (klešteno na ±100 %)
```

5.1 Jedinice — šta je float, šta je int

Sve u regulatoru je `float`, i to je ispravno:

veličina	jedinica	zašto float
reference, feedback, error	rad/s	brzina točka nije ceo broj (npr. 12.37 rad/s)
command	% maksimalnog PWM-a, −100..+100	regulator ne zna za CCR — to je posao motors.c
Kp	% po rad/s	pojačanje
Ki	% po rad/s, po prolazu	T je uračunato (kao u PIC32 kodu)
Kd	% po (rad/s po prolazu)	isto

Ceo broj se pojavljuje **tek u poslednjem koraku**: `motors.c` pretvori procenat u ispunu, a tvoj `motor.c` ispunu u CCR registar (`uint32_t`). Do tada nema zaokruživanja, pa nema ni gubitka rezolucije.

5.2 Strukture

Core/Src/Lib/PID.h

```c
#ifndef LIB_PID_H_
#define LIB_PID_H_
#include <stdbool.h>

/* Zajednicki koeficijenti za oba tocka. Menjaju se preko CAN-a (PID_KP=103, PID_KI=104, PID_KD=105). */
typedef struct {
    volatile float proportional_gain;              /* Kp */
    volatile float integral_gain;                  /* Ki, po tiku */
    volatile float derivative_gain;                /* Kd, po tiku */
    volatile float derivative_filter_coefficient;  /* a = e^(-2*pi*fc*T), 0..1 */
} pid_gains_t;

typedef struct {
    const pid_gains_t *gains;                      /* zajednicki koeficijenti */

    volatile float reference;                      /* zadata brzina tocka [rad/s] */
    volatile float feedback;                       /* izmerena brzina tocka [rad/s] */
    volatile float error;                          /* reference - feedback [rad/s] */
    volatile float error_previous;                 /* greska iz prethodnog tika */

    volatile float integrator;                     /* akumulirani I clan [%] */
    volatile float integrator_limit;               /* +- granica I clana [%] */

    volatile float derivative_raw;                 /* error - error_previous */
    volatile float derivative_filtered;            /* posle niskopropusnog filtera */

    volatile float command;                        /* izlaz regulatora [% maks. PWM-a] */
    volatile float command_limit;                  /* +- granica izlaza [%] */
} wheel_pid_t;

/* Podesi zajednicke koeficijente. derivative_filter_fc_hz i control_freq_hz
 * sluze da se izracuna koeficijent filtera D clana. */
void wheel_pid_gains_init(pid_gains_t *gains, float kp, float ki, float kd,
                          float derivative_filter_fc_hz, float control_freq_hz);

/* Poveze regulator sa zajednickim koeficijentima, postavi granice, obrise stanje. */
void wheel_pid_init(wheel_pid_t *pid, const pid_gains_t *gains,
                    float command_limit_pct, float integrator_limit_pct);

/* Obrise stanje (greska, I clan, D filter, izlaz) - koeficijenti ostaju. */
void wheel_pid_reset(wheel_pid_t *pid);

/* Jedan korak regulatora. Pozivati tacno control_freq_hz puta u sekundi.
 * Vraca command u % (-100..+100). */
float wheel_pid_update(wheel_pid_t *pid, float reference, float feedback);

#endif /* LIB_PID_H_ */
```

polje	čemu služi
`pid_gains_t`	**zajednički koeficijenti**. Postoji jedan primerak (`wheel_gains` u main.c), oba regulatora drže pokazivač na njega. CAN menja samo taj jedan skup — nemoguće je da levi i desni imaju različita pojačanja.
`derivative_filter_coefficient`	ono a = e^(−2π·fc·T) za D član; računa se jednom pri inicijalizaciji
`reference`, `feedback`	zadata i izmerena brzina [rad/s]; čuvaju se da bi se videle u debageru
`error`, `error_previous`	greška sada i u prošlom prolazu — razlika je sirovi izvod

polje	čemu služi
`integrator`, `integrator_limit`	akumulirani I član [%] i njegova granica
`derivative_raw`, `derivative_filtered`	izvod pre i posle filtera
`command`, `command_limit`	izlaz [%] i njegova granica
sve `volatile`	strukturu piše TIM7 prekid, a čitaju je i menjaju CAN prekid i debager — `volatile` tera kompajler da svaki put stvarno čita iz memorije

5.3 `wheel_pid_update()` — srce regulacije, red po red

Jedina funkcija koja stvarno računa. Poziva se 500 puta u sekundi, dva puta (za svaki točak).

PID.c — wheel_pid_update()

```
35  float wheel_pid_update(wheel_pid_t *pid, float reference, float feedback)
36  {
37      const pid_gains_t *g = pid->gains;
38
39      pid->reference      = reference;
40      pid->feedback       = feedback;
41      pid->error_previous = pid->error;
42      pid->error          = reference - feedback;
43
44      /* P clan */
45      float proportional_term = g->proportional_gain * pid->error;
46
47      /* I clan - prvo se izracuna novi, klesti, a prihvata tek ako nema windup-a */
48      float integrator_new = pid->integrator + g->integral_gain * pid->error;
49      if (integrator_new >  pid->integrator_limit) integrator_new =  pid->integrator_limit;
50      if (integrator_new < -pid->integrator_limit) integrator_new = -pid->integrator_limit;
51
52      /* D clan - izvod greske po tiku, kroz niskopropusni filter prvog reda.
53       * Filtrira se izvod, pa se tek onda mnozi sa Kd, da promena Kd preko
54       * CAN-a ne ostavi "rep" u stanju filtera. */
55      pid->derivative_raw      = pid->error - pid->error_previous;
56      pid->derivative_filtered = lowpass_step(g->derivative_filter_coefficient,
57                                              pid->derivative_filtered,
58                                              pid->derivative_raw);
59      float derivative_term = g->derivative_gain * pid->derivative_filtered;
60
61      float command = proportional_term + integrator_new + derivative_term;
62
63      /* Klestenje izlaza + anti-windup: ako je izlaz u zasicenju i greska gura
64       * dalje u zasicenje, I clan se ne pomera u ovom tiku. */
65      bool integrator_ok = true;
66      if (command > pid->command_limit) {
67          command = pid->command_limit;
68          if (pid->error > 0.0f) integrator_ok = false;
69      } else if (command < -pid->command_limit) {
70          command = -pid->command_limit;
71          if (pid->error < 0.0f) integrator_ok = false;
72      }
73      if (integrator_ok)
74          pid->integrator = integrator_new;
75
76      pid->command = command;
77      return command;
78  }
```

kod	šta radi
`const pid_gains_t *g = pid->gains;`	Uzmi pokazivač na zajedničke koeficijente, da se dalje piše kraće.
`pid->reference = reference;` `pid->feedback  = feedback;`	Zapamti ulaze. Ne koriste se u računu — stoje da bi mogao da ih vidiš u debageru.
`pid->error_previous = pid->error;` `pid->error = reference - feedback;`	Stara greška ide u `error_previous`, pa se računa nova. Pozitivna = vozimo presporo, negativna = prebrzo. Redosled je bitan — prvo sačuvaj, pa piši.
`float proportional_term =` `  g->proportional_gain * pid->error;`	**P član.** Sa Kp = 5.7, greška od 10 rad/s daje 57 % PWM-a.
`float integrator_new = pid->integrator` `  + g->integral_gain * pid->error;`	**I član** — ali u privremenu promenljivu. Da li će se stvarno upisati, odlučuje se tek na kraju (anti-windup).

`if (integrator_new >  pid->integrator_limit)`
`    integrator_new =  pid->integrator_limit;`
`if (integrator_new < -pid->integrator_limit)`
`    integrator_new = -pid->integrator_limit;`
	Klještenje I člana na ±100 %. Prva brana protiv raspumpavanja.

`pid->derivative_raw =` `  pid->error - pid->error_previous;`	**Sirovi izvod** — koliko se greška promenila od prošlog prolaza. Pri kvantizovanoj brzini ovo skače ±0.38 rad/s bez ikakvog razloga, zato ide u filter.

`pid->derivative_filtered = lowpass_step(`
`  g->derivative_filter_coefficient,`
`  pid->derivative_filtered,`
`  pid->derivative_raw);`
	Filter na izvodu. Filtrira se **izvod**, pa se tek onda množi sa Kd — da promena Kd preko CAN-a ne ostavi „rep" starih vrednosti u stanju filtera.

`float derivative_term =` `  g->derivative_gain * pid->derivative_filtered;`	**D član.** Sa Kd = 0 (što ros2 šalje) ovo je nula.
`float command = proportional_term` `  + integrator_new + derivative_term;`	Zbir sva tri.

`bool integrator_ok = true;`
`if (command > pid->command_limit) {`
`  command = pid->command_limit;`
`  if (pid->error > 0.0f)`
`    integrator_ok = false;`
`}`
	Anti-windup. Ako je izlaz probio +100 %, klešti ga. I ako greška i dalje traži „više" (pozitivna je), integrator se u ovom prolazu **ne menja** — nema svrhe da raste kad motor ionako ne može više. Isto simetrično za −100 %.

`if (integrator_ok)` `    pid->integrator = integrator_new;`	Tek sada se novi I član stvarno upisuje — samo ako nije bilo windup-a.
`pid->command = command;` `return command;`	Sačuvaj i vrati komandu. Ide u `PWM_SetCommand_*`.

Zašto anti-windup uopšte

Zamisli da ros2 traži 40 rad/s, a motor na 18 V može najviše 29. Greška je stalno pozitivna. Bez zaštite bi integrator rastao bez kraja — posle 10 sekundi bio bi na hiljadama procenata. Kad ros2 onda spusti referencu na 20 rad/s, regulator bi držao pun gas dok se integrator ne „odmota", i robot bi jurio sekundama. Sa ovom zaštitom, simulacija pokazuje oporavak za **14 ms**.

5.4 Ostale funkcije

PID.c — wheel_pid_gains_init()

```
4  void wheel_pid_gains_init(pid_gains_t *gains, float kp, float ki, float kd,
5                            float derivative_filter_fc_hz, float control_freq_hz)
6  {
7      gains->proportional_gain = kp;
8      gains->integral_gain     = ki;
9      gains->derivative_gain   = kd;
10     gains->derivative_filter_coefficient =
11         lowpass_coefficient(derivative_filter_fc_hz, control_freq_hz);
12 }
```

PID.c — wheel_pid_init()

```
14 void wheel_pid_init(wheel_pid_t *pid, const pid_gains_t *gains,
15                     float command_limit_pct, float integrator_limit_pct)
16 {
17     pid->gains            = gains;
18     pid->command_limit    = command_limit_pct;
19     pid->integrator_limit = integrator_limit_pct;
20     wheel_pid_reset(pid);
21 }
```

PID.c — wheel_pid_reset()

```
23 void wheel_pid_reset(wheel_pid_t *pid)
24 {
25     pid->reference           = 0.0f;
26     pid->feedback            = 0.0f;
27     pid->error               = 0.0f;
28     pid->error_previous      = 0.0f;
29     pid->integrator          = 0.0f;
30     pid->derivative_raw      = 0.0f;
31     pid->derivative_filtered = 0.0f;
32     pid->command             = 0.0f;
33 }
```

kod	šta radi

`void wheel_pid_gains_init(`
`  pid_gains_t *gains,`
`  float kp, float ki, float kd,`
`  float derivative_filter_fc_hz,`
`  float control_freq_hz)`
	Popunjava zajednički skup: tri pojačanja i koeficijent filtera D člana (računa ga iz fc i frekvencije petlje). Zove se jednom, u `main()`, **pre** nego što se regulatori povežu.

`void wheel_pid_init(`
`  wheel_pid_t *pid,`
`  const pid_gains_t *gains,`
`  float command_limit_pct,`
`  float integrator_limit_pct)`
	Poveže regulator sa koeficijentima (samo pokazivač, ne kopija), upiše granice, i obriše stanje.

`void wheel_pid_reset(` `  wheel_pid_t *pid)`	Nulira **stanje** (greške, integrator, D filter, izlaz) ali **ne dira koeficijente**. Zato posle `MOTORS_ON` ili `RESET_PID` ne moraš ponovo da šalješ Kp, Ki, Kd.

6. encoders.h / encoders.c — čitanje enkodera

Najmanji fajl u projektu. Tajmeri TIM2 i TIM3 su u CubeMX-u podešeni u **encoder modu** — hardver sam broji impulse i sam zna smer, procesor samo pročita brojač. Nema prekida, nema računanja.

Core/Src/Lib/encoders.c

```c
#include "encoders.h"

void encoder_init(void)
{
    HAL_TIM_Encoder_Start(&ENC_LEFT_TIMER, TIM_CHANNEL_ALL);
    HAL_TIM_Encoder_Start(&ENC_RIGHT_TIMER, TIM_CHANNEL_ALL);
    encoder_reset();
}

void encoder_reset(void)
{
    ENC_LEFT_TIMER.Instance->CNT = 0;
    ENC_RIGHT_TIMER.Instance->CNT = 0;
}

int32_t encoder_get_count_left_motor(void)
{
    return (int32_t)ENC_LEFT_TIMER.Instance->CNT;
}

int32_t encoder_get_count_right_motor(void)
{
    return (int32_t)ENC_RIGHT_TIMER.Instance->CNT;
}
```

kod	šta radi
`void encoder_init(void) { HAL_TIM_Encoder_Start( &ENC_LEFT_TIMER, TIM_CHANNEL_ALL); HAL_TIM_Encoder_Start( &ENC_RIGHT_TIMER, TIM_CHANNEL_ALL); encoder_reset(); }`	Pušta oba tajmera da broje. `TIM_CHANNEL_ALL` je obavezno — u encoder modu se koriste oba kanala (A i B), jedan daje impulse, drugi smer. Na kraju nulira brojače da se kreće od čistog stanja.
`void encoder_reset(void) { ENC_LEFT_TIMER.Instance->CNT = 0; ENC_RIGHT_TIMER.Instance->CNT = 0; }`	Upisuje nulu direktno u hardverski registar brojača. Zove se pri pokretanju i kad stigne CAN poruka 115.
`int32_t encoder_get_count_left_motor(void) { return (int32_t) ENC_LEFT_TIMER.Instance->CNT; }`	Vraća trenutnu vrednost brojača. Levi (TIM2) je 32-bitni, desni (TIM3) **16-bitni i stalno se preliva** — kako se to obrađuje, objašnjeno je u sledećem poglavlju.

7. odometry.h / odometry.c — od tikova do brzine u rad/s

Ime fajla je ostalo „odometrija", ali na STM-u se od nje koristi samo ono što regulatoru i ros2-u treba: **razlika brojača po prolazu**, **ukupni tikovi** (int32, za 102) i **brzina točka u rad/s**, sirova i filtrirana. Računanje x, y i ugla je unutra ali isključeno — vidi 7.6.

7.1 Zaglavlje

Core/Src/Lib/odometry.h

```
#ifndef LIB_ODOMETRY_H_
#define LIB_ODOMETRY_H_

#include "board_config.h"
#include "encoders.h"
#include <stdint.h>

void odometry_init(void);      /* posle encoder_init(); racuna koeficijent filtera */
void odometry_reset(void);     /* nula brojace, brzine i filter (i pozu ako je ukljucena) */

/* Pozvati tacno CONTROL_FREQ_HZ puta u sekundi (iz TIM7 prekida). */
void odometry_update(void);

float get_wheel_speed_left(void);       /* rad/s, filtrirano - za PID i za CAN */
float get_wheel_speed_right(void);
float get_wheel_speed_left_raw(void);   /* rad/s, iz jednog tika, nefiltrirano - debug */
float get_wheel_speed_right_raw(void);

int32_t get_encoder_total_left(void);   /* ukupni tickovi od reseta (CAN 102) */
int32_t get_encoder_total_right(void);

#if ODOMETRY_ON_STM
float get_x(void);
float get_y(void);
float get_fi(void);
float get_unwrapped_fi(void);
void  set_x_y(float x_p, float y_p, float fi_p);
#endif

#endif /* LIB_ODOMETRY_H_ */
```

7.2 Promenljive fajla

```
static int32_t last_count_left, last_count_right;    /* brojač iz prošlog prolaza */

static volatile int32_t delta_ticks_left, delta_ticks_right;  /* tikovi u poslednjem prolazu */
static volatile int32_t total_ticks_left, total_ticks_right;  /* ukupno od reseta, int32 */

static volatile float speed_left_raw, speed_right_raw;  /* rad/s iz jednog prolaza */
static volatile float speed_left,     speed_right;      /* rad/s posle filtera */

static float speed_filter_coefficient;                  /* a = e^(-2·π·fc·T) */
```

promenljiva	objašnjenje
`last_count_*`	vrednost brojača kad smo poslednji put gledali. Bez ovoga ne bismo znali *koliko se promenilo*, a samo to nas zanima. Nije volatile — dira je samo prekid.
`delta_ticks_*`	koliko tikova u poslednjih 2 ms. Za debug.

promenljiva	objašnjenje
`total_ticks_*`	**32-bitni** zbir svih tikova od reseta. Ovo ide u poruku 102 i ne preliva se kao hardverski brojač (2^{31} tikova = 262 000 obrtaja = 67 km).
`speed_*_raw`	brzina iz jednog prolaza: tikovi × RAD_PER_TICK × 500. Kvantizovana na 0.38 rad/s. Za debug i poređenje.
`speed_*`	**filtrirana** brzina. Ovo ide i u PID i na CAN.
`speed_filter_coefficient`	a za filter brzine, računa se jednom u `odometry_init()`

7.3 `odometry_init()` i `odometry_reset()`

odometry.c — odometry_init()

```
29  void odometry_init(void)
30  {
31      speed_filter_coefficient = lowpass_coefficient(SPEED_LPF_FC_HZ, (float)CONTROL_FREQ_HZ);
32      odometry_reset();
33  }
```

odometry.c — odometry_reset()

```
35  void odometry_reset(void)
36  {
37      encoder_reset();
38      last_count_left  = encoder_get_count_left_motor();
39      last_count_right = encoder_get_count_right_motor();
40
41      delta_ticks_left  = 0;   delta_ticks_right = 0;
42      total_ticks_left  = 0;   total_ticks_right = 0;
43      speed_left_raw    = 0.0f; speed_right_raw  = 0.0f;
44      speed_left        = 0.0f; speed_right      = 0.0f;
45
46  #if ODOMETRY_ON_STM
47      set_x_y(0.0f, 0.0f, 0.0f);
48  #endif
49  }
```

kod	šta radi
`speed_filter_coefficient = lowpass_coefficient(SPEED_LPF_FC_HZ, (float)CONTROL_FREQ_HZ);`	Izračunaj a za 30 Hz na 500 Hz: 0.686. Jednom, na startu.
`encoder_reset(); last_count_left = encoder_get_count_left_motor(); last_count_right = encoder_get_count_right_motor();`	Nuliraj hardverske brojače pa upiši zatečeno stanje kao polaznu tačku. Bez ovoga bi prvi prolaz izračunao ogroman lažni pomeraj.
`delta_ticks_* = 0; total_ticks_* = 0; speed_*_raw = 0.0f; speed_* = 0.0f;`	Sve na nulu, uključujući i stanje filtera — da posle reseta ne ostane „rep" stare brzine.
`#if ODOMETRY_ON_STM set_x_y(0.0f, 0.0f, 0.0f); #endif`	Nuliranje pozicije postoji samo kad je odometrija uključena; inače se ova linija ne kompajlira.

7.4 `odometry_update()` — poziva se 500 puta u sekundi

odometry.c — odometry_update()

```
51  void odometry_update(void)
52  {
53      int32_t count_left  = encoder_get_count_left_motor();
54      int32_t count_right = encoder_get_count_right_motor();
55
56      /* Razlika se racuna u 16 bita pa prosiruje sa znakom. To radi i za
57       * 32-bitni TIM2 i za 16-bitni TIM3, i prelazak brojaca preko nule ne
58       * pravi skok (dokle god je manje od 32768 tickova po tiku, a to je
59       * nemoguce - to bi bilo 4 obrtaja za 2 ms). */
60      int32_t dl = (int16_t)((uint16_t)count_left  - (uint16_t)last_count_left)  * ENC_LEFT_SIGN;
61      int32_t dr = (int16_t)((uint16_t)count_right - (uint16_t)last_count_right) * ENC_RIGHT_SIGN;
62
63      last_count_left  = count_left;
64      last_count_right = count_right;
65
66      delta_ticks_left  = dl;
67      delta_ticks_right = dr;
68      total_ticks_left  += dl;
69      total_ticks_right += dr;
70
71      /* tickovi po tiku -> rad/s: d * (2*pi / ticks_per_rev) * f_petlje */
72      speed_left_raw  = (float)dl * RAD_PER_TICK * (float)CONTROL_FREQ_HZ;
73      speed_right_raw = (float)dr * RAD_PER_TICK * (float)CONTROL_FREQ_HZ;
74
75      speed_left  = lowpass_step(speed_filter_coefficient, speed_left,  speed_left_raw);
76      speed_right = lowpass_step(speed_filter_coefficient, speed_right, speed_right_raw);
77
78  #if ODOMETRY_ON_STM
79      /* ---- pozicija robota - iskljuceno, radi je ros2 ---------------------- */
80      float ds_l = (float)dl * RAD_PER_TICK * wheel_radius_mm;   /* mm */
81      float ds_r = (float)dr * RAD_PER_TICK * wheel_radius_mm;
82      float ds   = (ds_r + ds_l) / 2.0f;
83      float dfi  = (ds_l - ds_r) / wheel_base_mm;
84      float fi_mid = fi + (dfi / 2.0f);
85
86      x += ds * cosf(fi_mid);
87      y += ds * sinf(fi_mid);
88      fi += dfi;
89      unwrapped_fi += dfi;
90      while (fi >  M_PI) fi -= 2.0f * M_PI;
91      while (fi < -M_PI) fi += 2.0f * M_PI;
92  #endif
93  }
```

kod	šta radi
`int32_t count_left  = encoder_get_count_left_motor();` `int32_t count_right = encoder_get_count_right_motor();`	Očitaj oba brojača.

`int32_t dl = (int16_t)((uint16_t)count_left`
`- (uint16_t)last_count_left)`
`* ENC_LEFT_SIGN;`
	Najvažniji red u fajlu. Razlika brojača izračunata tako da radi i kad se brojač prelije (objašnjenje ispod), pa pomnožena znakom iz konfiguracije.

`last_count_left  = count_left;` `last_count_right = count_right;`	Zapamti za sledeći prolaz.
`delta_ticks_left  = dl;` `total_ticks_left  += dl;`	Sačuvaj pomeraj i dodaj na ukupni 32-bitni zbir — to je ono što ros2 dobija na 102.
`speed_left_raw = (float)dl * RAD_PER_TICK` `* (float)CONTROL_FREQ_HZ;`	**Tikovi → rad/s.** Tikovi × (2π/8192) = radijani u ovih 2 ms; × 500 = radijani u sekundi. Na 10 rad/s prođe 26 tikova po prolazu.

kod	šta radi

`speed_left = lowpass_step(`
`  speed_filter_coefficient,`
`  speed_left, speed_left_raw);`
	Filter. Isti kao za D član, sa svojom fc (30 Hz). Ovo je vrednost koju vidi regulator i koju ros2 dobija na 101.

`#if ODOMETRY_ON_STM`
`  ... x, y, fi ...`
`#endif`
	Računanje pozicije — isključeno, ne kompajlira se (7.6).

Kako radi trik sa 16-bitnom razlikom

Desni brojač (TIM3) ide od 0 do 65535 pa se vrati na 0. Kod 8192 tikova po obrtaju to je **svakih 8 obrtaja točka** — dakle stalno. Levi (TIM2) je 32-bitni, ali kod ga namerno tretira isto, da ista logika radi za oba.

Da je pisano naivno, kao `d = current - last`, na prelivanju bi ispalo:

```
last = 65500,  current = 30     (točak je otišao 66 tikova napred)
naivno:  30 - 65500 = -65470       <-- robot "skočio" 8 obrtaja unazad!
```

Zato se radi ovako: oduzimanje se izvede u **neoznačenoj 16-bitnoj aritmetici**, gde se prelivanje samo poništi, a rezultat se zatim pročita kao **označen** 16-bitni broj:

```
(uint16_t)30 - (uint16_t)65500  =  66        (uint16 se "obmota")
(int16_t)66                     =  66       <-- tačno, 66 tikova napred
```

A ako se točak stvarno okretao unazad, isti izraz da negativan broj:

```
last = 30,  current = 65500
(uint16_t)65500 - (uint16_t)30 = 65470
(int16_t)65470                 = -66       <-- tačno, 66 tikova unazad
```

> Jedini uslov je da se točak **između dva očitavanja ne pomeri više od 32767 tikova**. Na 500 Hz i najvećoj brzini to je oko 77 tikova — 400 puta manje od granice. Simulacija: posle 22.8 m vožnje i 11 prelivanja, oba zbira su bila **tačna u tik**; u 75 nasumičnih scenarija sa nasumičnim početnim vrednostima brojača, najveća greška je bila 2 tika (jedan zaostali tik pri zaustavljanju).

7.5 Zašto filter, a ne prosek kao ranije

Ranija verzija je regulatoru davala sirovu brzinu, a na CAN slala prosek preko 5 prolaza. Sada obe strane dobijaju istu filtriranu brzinu. Rezultat na ustaljenih 15 rad/s (simulacija sa kvantizacijom enkodera):

način	rasipanje (σ)	ko ga koristi
sirovo, jedan prolaz	0.122 rad/s	nikad (samo debug)
prosek 5 prolaza (staro)	0.038 rad/s	samo CAN — regulator je gledao sirovo
filter 30 Hz (sada)	**0.038 rad/s**	**i regulator i CAN**
filter 15 Hz	0.020 rad/s	ako treba još mirnije (ali kasni ~10 ms)

Isto rasipanje kao stari prosek, ali kontinuirano (nije vezano za prozor od 10 ms) i, što je bitnije, **regulator više ne gleda sirovi šum** — što je posebno važno ako se ikad uključi D član.

7.6 Isključeni deo — pozicija robota

Ovo je kod koji stoji iza `#if ODOMETRY_ON_STM` i **ne ulazi u program**:

```
#if ODOMETRY_ON_STM
    float ds_l = (float)dl * RAD_PER_TICK * wheel_radius_mm;   /* mm */
    float ds_r = (float)dr * RAD_PER_TICK * wheel_radius_mm;
    float ds   = (ds_r + ds_l) / 2.0f;
    float dfi  = (ds_l - ds_r) / wheel_base_mm;
    float fi_mid = fi + (dfi / 2.0f);

    x += ds * cosf(fi_mid);
    y += ds * sinf(fi_mid);
    fi += dfi;
    unwrapped_fi += dfi;
    while (fi >  M_PI) fi -= 2.0f * M_PI;
    while (fi < -M_PI) fi += 2.0f * M_PI;
#endif
```

Isto važi za `get_x()`, `get_y()`, `get_fi()`, `get_unwrapped_fi()` i `set_x_y()` — deklarisani su i definisani samo kad je prekidač na 1. Zadržano je da se ne izgubi (radilo je), ali ros2 to radi iz enkodera koje mu šaljemo, sa svojim dimenzijama robota.

7.7 Getteri

kod	šta radi
`float get_wheel_speed_left(void)` `float get_wheel_speed_right(void)`	**Filtrirana** brzina u rad/s. Koriste je regulaciona petlja i feedback.
`float get_wheel_speed_left_raw(void)` `float get_wheel_speed_right_raw(void)`	Nefiltrirana, iz jednog prolaza. Samo za poređenje u debageru.
`int32_t get_encoder_total_left(void)` `int32_t get_encoder_total_right(void)`	Ukupni tikovi od reseta — to ide u poruku 102.

8. motors.h / motors.c — od procenta do PWM-a

Ovaj fajl je **tanak sloj** iznad tvojih postojećih `motor.c` i `motor2.c`. Ti fajlovi nisu menjani — oni znaju kako se postavlja CCR registar i kako se gura DIR pin. Ovaj sloj im govori *koliko* i *u kom smeru*, i pazi da se ne pređe 18 V.

	levi točak	desni točak
tvoje funkcije	`Motor_*`	`Motor2_*`
PWM pin	PE9 (TIM1_CH1)	PE11 (TIM1_CH2)
DIR pin	PD9	PA8
nSLEEP pin	PD11	PA10
napred je	`Motor_SetDir(0)` — **obrnuto**	`Motor2_SetDir(1)`

> Levi motor ima **invertovanu DIR logiku** jer je fizički zalemljen obrnuto. To je ispravljeno softverski, u jednoj funkciji, umesto prelemljivanja. Ako ikad prelemiš motor, samo zameni 0 i 1 u `PWM_SetCommand_Left()`.

8.1 Zaglavlje

Core/Src/Lib/motors.h

```
14  #ifndef LIB_MOTORS_H_
15  #define LIB_MOTORS_H_
16  #include "board_config.h"
17  #include <stdbool.h>
18
19  /* Napon napajanja motora [V]. Globalna promenljiva, ne makro - kasnije ce se
20   * na startu ucitati sa ADC-a preko PWM_SetSupplyVoltage(). Dok se to ne
21   * uradi, vazi MOTOR_SUPPLY_VOLTAGE_DEFAULT. */
22  extern volatile float motor_supply_voltage;
23
24  void  PWM_Init(void);
25  void  PWM_Enable(bool on);            /* nSLEEP oba drajvera + duty 0 */
26  void  PWM_Stop(void);                 /* duty 0 na oba, drajveri ostaju budni */
27
28  void  PWM_SetCommand_Left(float percent);   /* -100..+100 % maks. PWM-a */
29  void  PWM_SetCommand_Right(float percent);
30
31  /* Upisi izmereni napon napajanja i preracunaj plafon ispune.
32   * Vraca false (i ne menja nista) ako je vrednost van MOTOR_SUPPLY_VOLTAGE_MIN..MAX. */
33  bool  PWM_SetSupplyVoltage(float volts);
34  float PWM_GetMaxDutyPct(void);        /* trenutni plafon ispune [%], za watch */
35
36  #endif /* LIB_MOTORS_H_ */
```

8.2 Napon napajanja i plafon ispune

```
volatile float motor_supply_voltage = MOTOR_SUPPLY_VOLTAGE_DEFAULT;

static volatile float max_duty_pct = MOTOR_MAX_VOLTAGE / MOTOR_SUPPLY_VOLTAGE_DEFAULT * 100.0f;
```

`motor_supply_voltage` je **globalna promenljiva**, vidljiva svima preko `motors.h`. Sada je 30 V i niko je ne menja. `max_duty_pct` je plafon ispune koji iz nje sledi — 60 %. Obe su `volatile` jer ih piše glavna petlja (ili budući ADC kod), a čita TIM7 prekid.

motors.c — supply_voltage_valid()

```
18  static bool supply_voltage_valid(float v)
19  {
20      /* "v >= MIN && v <= MAX" je false i za NaN, pa NaN ne prolazi */
21      return (v >= MOTOR_SUPPLY_VOLTAGE_MIN) && (v <= MOTOR_SUPPLY_VOLTAGE_MAX);
22  }
```

motors.c — recompute_max_duty()

```
24  static void recompute_max_duty(void)
25  {
26      float v = motor_supply_voltage;
27      if (!supply_voltage_valid(v)) {
28          v = MOTOR_SUPPLY_VOLTAGE_DEFAULT;      /* neko je upisao glupost - nazad na sigurno */
29          motor_supply_voltage = v;
30      }
31      float d = MOTOR_MAX_VOLTAGE / v * 100.0f;
32      if (d > 100.0f) d = 100.0f;
33      max_duty_pct = d;
34  }
```

motors.c — PWM_SetSupplyVoltage()

```
36  bool PWM_SetSupplyVoltage(float volts)
37  {
38      if (!supply_voltage_valid(volts))
39          return false;
40      motor_supply_voltage = volts;
41      recompute_max_duty();
42      return true;
43  }
```

kod	šta radi
`return (v >= MOTOR_SUPPLY_VOLTAGE_MIN) && (v <= MOTOR_SUPPLY_VOLTAGE_MAX);`	Napon mora da bude između 12 i 40 V. Poređenje sa NaN je uvek `false`, pa NaN ne prolazi ni ovde.
`if (!supply_voltage_valid(v)) { v = MOTOR_SUPPLY_VOLTAGE_DEFAULT; motor_supply_voltage = v; }`	Ako je neko direktno upisao glupost u globalnu promenljivu, vrati na 30 V. Sigurna strana: 30 V je najveći realan napon, pa daje **najniži** plafon.
`float d = MOTOR_MAX_VOLTAGE / v * 100.0f; if (d > 100.0f) d = 100.0f; max_duty_pct = d;`	18 / V × 100. Na 30 V = 60 %, na 24 V = 75 %, na 18 V i niže = 100 % (ne može više). Motor uvek dobija tačno 18 V pri punoj komandi, koliko god baterija bila puna.
`bool PWM_SetSupplyVoltage(float volts) { if (!supply_voltage_valid(volts)) return false; motor_supply_voltage = volts; recompute_max_duty(); return true; }`	Ulazna tačka za ADC. Neispravna vrednost se **odbija** i ništa se ne menja; vraća `false` da pozivalac zna.

napon napajanja	plafon ispune	CCR max (od 2560)	na motoru pri 100 %
30 V (sada)	**60.0 %**	**1536**	18.0 V
28 V	64.3 %	1645	18.0 V
24 V	75.0 %	1920	18.0 V

napon napajanja	plafon ispune	CCR max (od 2560)	na motoru pri 100 %
20 V	90.0 %	2304	18.0 V
18 V i niže	100 %	2560	= napon baterije

8.3 `command_to_duty()` — poslednja brana pred CCR registrom

motors.c — command_to_duty()

```
75  static float command_to_duty(float percent)
76  {
77      if (!(percent > 0.0f))   percent = 0.0f;
78      if (percent > 100.0f)    percent = 100.0f;
79
80      float duty = (percent / 100.0f) * max_duty_pct;
81      if (duty > max_duty_pct) duty = max_duty_pct;
82      return duty;
83  }
```

kod	šta radi
`if (!(percent > 0.0f))  percent = 0.0f;`	Namerno napisano kao „nije veće od nule" umesto „manje od nule": **poređenje sa NaN je uvek false**, pa NaN ovde postane 0. Da je pisano `if (percent < 0)`, NaN bi prošao dalje, a `(uint32_t)NaN` u motor.c je nedefinisan i može da da CCR veći od ARR-a = trajno 100 % = 30 V na motoru.
`if (percent > 100.0f)  percent = 100.0f;`	Regulator ionako ne daje više od 100, ali ako neko kasnije pozove funkciju direktno, i dalje je bezbedno.
`float duty = (percent / 100.0f) * max_duty_pct;`	**Ovde je zaštita od prenapona.** Procenat komande se skalira na plafon: 100 % komande = 60 % ispune (pri 30 V). Nikad se ne množi sa 100.
`if (duty > max_duty_pct) duty = max_duty_pct;` `return duty;`	Još jedno klještenje, čisto da ne zavisi od tačnosti float aritmetike iznad.

Šta se dešava sa brojevima (pri 30 V)

komanda regulatora	ispuna	CCR (od 2560)	napon na motoru
0 %	0 %	0	0 V
25 %	15 %	384	4.5 V
50 %	30 %	768	9 V
100 %	**60 %**	**1536**	**18 V**
150 % (nemoguće, ali)	60 %	1536	18 V
NaN	0 %	0	0 V

8.4 `PWM_SetCommand_Left()` i `PWM_SetCommand_Right()`

motors.c — PWM_SetCommand_Left()

```
85  void PWM_SetCommand_Left(float percent)
86  {
87      if (percent >= 0.0f) {
88          Motor_SetDir(0);            /* levi je invertovan: 0 = napred */
89      } else {
90          Motor_SetDir(1);
91          percent = -percent;
92      }
93      Motor_SetDuty(command_to_duty(percent));
94  }
```

motors.c — PWM_SetCommand_Right()

```
96  void PWM_SetCommand_Right(float percent)
97  {
98      if (percent >= 0.0f) {
99          Motor2_SetDir(1);           /* desni: 1 = napred */
100     } else {
101         Motor2_SetDir(0);
102         percent = -percent;
103     }
104     Motor2_SetDuty(command_to_duty(percent));
105 }
```

kod	šta radi
`if (percent >= 0.0f) { Motor_SetDir(0); } else { Motor_SetDir(1); percent = -percent; }`	Razdvaja **smer** od **jačine**. Znak komande određuje DIR pin, a posle toga se radi samo sa pozitivnim brojem. Levi: 0 = napred (invertovan), desni: 1 = napred.
`Motor_SetDuty(command_to_duty(percent));`	Jačina kroz branu iz 8.3, pa u tvoj motor.c koji upisuje CCR.

Pri 60 % ispune motor fizički vozi oko **29.4 rad/s**. Ako ros2 traži više, regulator ulazi u zasićenje na 100 % i brzina ostaje na tih 29.4 — anti-windup u PID-u se brine da to ne ostavi posledice.

8.5 Ostale funkcije

motors.c — PWM_Init()

```
50  void PWM_Init(void)
51  {
52      recompute_max_duty();
53      Motor_Init();
54      Motor2_Init();
55      Motor_Enable(1);
56      Motor2_Enable(1);
57  }
```

motors.c — PWM_Enable()

```
59  void PWM_Enable(bool on)
60  {
61      Motor_SetDuty(0.0f);
62      Motor2_SetDuty(0.0f);
63      Motor_Enable(on ? 1 : 0);
64      Motor2_Enable(on ? 1 : 0);
65  }
```

motors.c — PWM_Stop()

```
67  void PWM_Stop(void)
68  {
69      Motor_SetDuty(0.0f);
70      Motor2_SetDuty(0.0f);
71  }
```

kod	šta radi
`void PWM_Init(void) { recompute_max_duty(); Motor_Init(); Motor2_Init(); Motor_Enable(1); Motor2_Enable(1); }`	Poziva se jednom, iz `main()`. **Prvo** izračuna plafon iz napona (da bude spreman pre prvog PWM-a), pa tvoje `Motor_Init()` startuju PWM kanale na 0 %, pa se bude drajveri.
`void PWM_Enable(bool on) { Motor_SetDuty(0.0f); Motor2_SetDuty(0.0f); Motor_Enable(on); Motor2_Enable(on); }`	Za poruke MOTORS_ON / MOTORS_OFF. **Prvo nulira PWM pa tek onda dira nSLEEP** — da drajver ne bi budio sa zatečenim duty ciklusom i robot poskočio.
`void PWM_Stop(void) { Motor_SetDuty(0.0f); Motor2_SetDuty(0.0f); }`	Samo nulira PWM, drajveri ostaju budni. Koristi se u regulacionoj petlji dok su motori „ugašeni" softverski.
`float PWM_GetMaxDutyPct(void)`	Vraća trenutni plafon — da ga vidiš u debageru ili jednog dana pošalješ na CAN.

9. can_comm.h — protokol

Zaglavlje definiše **dogovor sa Raspberry stranom**. Ako se ovde nešto promeni, mora i tamo. Zato su ID-jevi prepisani iz ros2 enum-a `CanID` doslovno — ista imena, iste vrednosti:

Core/Src/Lib/can_comm.h — enum CanID

```
50  enum CanID {
51      WHEEL_COMMAND_VEL  = 100,
52      WHEEL_FEEDBACK_VEL = 101,
53      WHEEL_POSITION     = 102,
54      PID_KP             = 103,
55      PID_KI             = 104,
56      PID_KD             = 105,
57      PID_KP_ROT         = 106,
58      PID_KI_ROT         = 107,
59      PID_KD_ROT         = 108,
60      MOTORS_ON          = 109,
61      MOTORS_OFF         = 110,
62      FEEDBACK_ON        = 111,
63      FEEDBACK_OFF       = 112,
64      REFERENCE_ON       = 113,
65      REFERENCE_OFF      = 114,
66      RESET_ENCODERS     = 115,
67      RESET_PID          = 116,
68      FRONT_IR           = 150,
69      BACK_IR            = 151,
70      CINCH              = 152
71  };
```

Vrednosti su dekadne, ne heksadekadne. Ranije je u ovom kodu pisalo 0x100–0x116 i 0x150–0x152 — a to su brojevi 256–278 i 336–338, dakle sasvim drugi ID-jevi od onih koje ros2 šalje. Ploča i Raspberry se tako **nikad ne bi čuli**: STM ne bi primio nijednu komandu, a ros2 ne bi prepoznao nijedan feedback. Sada su usklađeni, a simulacija to i proverava (test T19).

9.1 Poruke koje STM prima

ID	sadržaj	šta radi
100	2 × float (rad/s)	zadata brzina levog i desnog točka — proverava se (NaN) i odseca na ±60
103	float	Kp — u zajednički skup, oba točka
104	float	Ki — isto
105	float	Kd — isto; **koristi se** (ros2 šalje 0 da ga isključi)
106–108	float	Kp/Ki/Kd **ugaonog** regulatora (krivolinijsko kretanje u staroj linear/angular šemi). Sa regulatorima po točku nema gde da se primeni — prima se i čuva.
109 / 110	—	MOTORS_ON / MOTORS_OFF
111 / 112	—	FEEDBACK_ON / FEEDBACK_OFF
113 / 114	—	REFERENCE_ON / REFERENCE_OFF
115 / 116	—	RESET_ENCODERS / RESET_PID

9.2 Poruke koje STM šalje

ID	sadržaj	kada
101	2 × float (rad/s)	izmerene **filtrirane** brzine, 100 Hz — **samo ako je feedback uključen**
102	2 × int32	ukupni tikovi enkodera, 100 Hz — isti uslov
150	uint8 (0/1)	prednji IR — samo pri promeni, 10 puta
151	uint8 (0/1)	zadnji IR — samo pri promeni, 10 puta
152	uint8 (0/1)	cinc — pri promeni 10 puta, pa ponavljanje svake sekunde

9.3 Važne konstante

kod	šta radi
`#define CAN_LITTLE_ENDIAN 0`	Redosled bajtova u brojevima. **0 = big-endian** (najviši bajt prvi), kao u referentnom PIC32 kodu. Ako Raspberry pakuje little-endian (`struct.pack('<f')`), stavi 1 i ceo kod se prilagodi. Ovo je najčešći uzrok da „stižu ludi brojevi".
`#define CAN_IR_BURST_COUNT 10` `#define CAN_CINC_BURST_COUNT 10`	Koliko puta se ponovi poruka pri promeni stanja. CAN nema potvrdu prijema na nivou aplikacije, pa se bitna jednokratna vest šalje više puta.
`#define CAN_CINC_HEARTBEAT_MS 1000`	Cinc se dodatno ponavlja svake sekunde, da bi i ploča koja se kasnije upali saznala da je meč već počeo. 0 gasi ovo ponavljanje.
`#define CAN_STARTUP_FLUSH_FRAMES 2000`	Koliko nula-poruka poslati na startu da se isprazni bafer na Linux strani.
`#define CAN_STARTUP_FLUSH_MAX_MS 2000`	Gornja granica koliko to sme da traje. **Bez ovoga**, ako Raspberry nije upaljen (niko ne potvrđuje poruke na busu), slanje bi blokiralo start ploče na desetak sekundi.
`#define CAN_STARTUP_RX_DRAIN_MS 1000`	Koliko dugo se na startu prazni sopstveni prijemni red.
`#define CAN_CMD_TIMEOUT_MS 500`	Ako pola sekunde ne stigne nijedna poruka 100, zadata brzina se nuliše. **Sigurnosna mera** — ako Raspberry crkne ili se otkači kabl, robot staje umesto da nastavi pravo. 0 isključuje.

10. can_comm.c — implementacija komunikacije

10.1 Pakovanje brojeva u bajtove

CAN prenosi samo bajtove. Float i int32 se moraju rastaviti na 4 bajta i sastaviti na drugoj strani, i **obe strane moraju da se slože oko redosleda**.

kod	šta radi

`static void pack_u32(uint8_t *b, uint32_t v)`
`{`
`#if CAN_LITTLE_ENDIAN`
`  b[0] = v & 0xFF;`
`  b[1] = (v >> 8) & 0xFF;`
`  ...`
`#else`
`  b[3] = v & 0xFF;`
`  b[2] = (v >> 8) & 0xFF;`
`  ...`
`#endif`
`}`
	Rastavlja 32-bitni broj na 4 bajta. Uslovno prevođenje znači da se u finalni program ugradi **samo jedna** od dve varijante — nema provere u toku rada, nema gubitka brzine.

`static void pack_float(uint8_t *b, float f)`
`{`
`  uint32_t u;`
`  memcpy(&u, &f, 4);`
`  pack_u32(b, u);`
`}`
	Uzima **bitove** float broja i tretira ih kao ceo broj, pa ih pakuje kao i svaki drugi. `memcpy` je ovde namerno — ono je siguran način da se pročitaju bitovi bez kršenja pravila jezika o tipovima (za razliku od `*(uint32_t*)&f`, koje kompajler sme da optimizuje pogrešno).

`static uint8_t dlc_to_len(uint32_t dlc)`
`{`
`  uint32_t v = (dlc > 15) ? (dlc >> 16) : dlc;`
`  return (v <= 8) ? (uint8_t)v : 8;`
`}`
	Pretvara HAL-ov zapis dužine poruke u običan broj bajtova. Različite verzije HAL-a to kodiraju različito (negde kao broj, negde pomereno za 16 bita) — ovo pokriva oba slučaja.

10.2 Provera brojeva sa CAN-a

can_comm.c — unpack_float_checked()

```
103  static bool unpack_float_checked(const uint8_t *b, float *out)
104  {
105      float f = unpack_float(b);
106      if (!isfinite(f))
107          return false;
108      *out = f;
109      return true;
110  }
```

can_comm.c — clamp_speed()

```
112  static float clamp_speed(float w)
113  {
114      if (w >  MAX_WHEEL_SPEED_RADS) return  MAX_WHEEL_SPEED_RADS;
115      if (w < -MAX_WHEEL_SPEED_RADS) return -MAX_WHEEL_SPEED_RADS;
116      return w;
117  }
```

kod	šta radi

`float f = unpack_float(b);`
`if (!isfinite(f))`
`  return false;`
`*out = f;`
`return true;`
	Samo konačan broj sme u regulator. Ako sa CAN-a stigne NaN ili beskonačno (pogrešan bajt, greška na ROS strani), integrator bi ga „upio" zauvek i prošao bi kroz svako klještenje (poređenje sa NaN je false). Ovako se cela poruka odbaci, a stara vrednost ostaje.

`if (w > MAX_WHEEL_SPEED_RADS) return MAX_WHEEL_SPEED_RADS;`
`if (w < -MAX_WHEEL_SPEED_RADS) return -MAX_WHEEL_SPEED_RADS;`
`return w;`
	Odsecanje zadate brzine na ±60 rad/s.

10.3 `can_send()` — slanje jedne poruke

can_comm.c — can_send()

```
145  static bool can_send(uint32_t id, const uint8_t *data, uint8_t len)
146  {
147      if (HAL_FDCAN_GetTxFifoFreeLevel(&hfdcan1) == 0)
148          return false;
149
150      FDCAN_TxHeaderTypeDef tx;
151      tx.Identifier          = id;
152      tx.IdType              = FDCAN_STANDARD_ID;
153      tx.TxFrameType         = FDCAN_DATA_FRAME;
154      tx.DataLength          = len_to_dlc(len);
155      tx.ErrorStateIndicator = FDCAN_ESI_ACTIVE;
156      tx.BitRateSwitch       = FDCAN_BRS_OFF;
157      tx.FDFormat            = FDCAN_CLASSIC_CAN;
158      tx.TxEventFifoControl  = FDCAN_NO_TX_EVENTS;
159      tx.MessageMarker       = 0;
160
161      return HAL_FDCAN_AddMessageToTxFifoQ(&hfdcan1, &tx, (uint8_t *)data) == HAL_OK;
162  }
```

kod	šta radi
`if (HAL_FDCAN_GetTxFifoFreeLevel(&hfdcan1) == 0)` `  return false;`	**Ako je red za slanje pun, odustani odmah.** Ovo je ključno: funkcija **nikad ne čeka**. Da čeka, jedna zagušena magistrala bi zaustavila celu glavnu petlju.

`tx.Identifier = id;`
`tx.IdType = FDCAN_STANDARD_ID;`
`tx.TxFrameType = FDCAN_DATA_FRAME;`
`tx.DataLength = len_to_dlc(len);`
	Popunjava zaglavlje: koji ID, standardni (11-bitni) format, obična podatkovna poruka, koliko bajtova.

`tx.BitRateSwitch = FDCAN_BRS_OFF;` `tx.FDFormat = FDCAN_CLASSIC_CAN;`	**Klasičan CAN, ne CAN-FD.** Mora tako jer je na drugoj strani običan CAN kontroler.
`return HAL_FDCAN_AddMessageToTxFifoQ(` `  &hfdcan1, &tx, data) == HAL_OK;`	Ubacuje poruku u red. Hardver je dalje sam šalje. Vraća `true` ako je primljena u red — pozivalac po tome zna da li da smanji brojač ponavljanja.

can_comm.c — send_speed_feedback()

```
164  static bool send_speed_feedback(void)
165  {
166      uint8_t buf[8];
167      /* ista filtrirana brzina [rad/s] koju koristi i PID */
168      pack_float(buf,     get_wheel_speed_left());
169      pack_float(buf + 4, get_wheel_speed_right());
170      return can_send(WHEEL_FEEDBACK_VEL, buf, 8);
171  }
```

can_comm.c — send_encoder_feedback()

```
173  static bool send_encoder_feedback(void)
174  {
175      uint8_t buf[8];
176      pack_i32(buf,     get_encoder_total_left());
177      pack_i32(buf + 4, get_encoder_total_right());
178      return can_send(WHEEL_POSITION, buf, 8);
179  }
```

Feedback funkcije pakuju filtriranu brzinu (rad/s, bez konverzije) i ukupne tikove.

10.4 `process_msg()` — obrada primljene poruke

Zove se iz prekida, za svaku pristiglu poruku. Velika `switch` naredba po ID-u.

can_comm.c — process_msg()

```
183 static void process_msg(uint32_t id, uint8_t len, const uint8_t *d)
184 {
185     switch (id)
186     {
187     case WHEEL_COMMAND_VEL:
188     {
189         if (len < 8) return;
190         last_cmd_tick = HAL_GetTick();
191         cmd_timed_out = false;
192         if (!can_reference_on) return;
193         /* ros2 salje rad/s, regulator radi u rad/s - samo provera i odsecanje */
194         float wl, wr;
195         if (!unpack_float_checked(d, &wl) || !unpack_float_checked(d + 4, &wr))
196             return;
197         can_ref_speed_left  = clamp_speed(wl);
198         can_ref_speed_right = clamp_speed(wr);
199         break;
200     }
201
202     /* Koeficijenti regulatora po tocku - jedan skup za oba tocka. */
203     case PID_KP:
204     {
205         float g;
206         if (len >= 4 && unpack_float_checked(d, &g)) wheel_gains.proportional_gain = g;
207         break;
208     }
209     case PID_KI:
210     {
211         float g;
212         if (len >= 4 && unpack_float_checked(d, &g)) wheel_gains.integral_gain = g;
213         break;
214     }
215     case PID_KD:
216     {
217         float g;
218         if (len >= 4 && unpack_float_checked(d, &g)) wheel_gains.derivative_gain = g;
219         break;
220     }
221
222     /* Ugaoni regulator (krivolinijsko kretanje) iz stare linear/angular seme.
223      * Sa regulatorima po tocku nema gde da se primeni - prima se i cuva da
224      * ros2 moze da salje bez greske i da se vidi u debageru. */
225     case PID_KP_ROT: if (len >= 4) can_kp_rot = unpack_float(d); break;
226     case PID_KI_ROT: if (len >= 4) can_ki_rot = unpack_float(d); break;
227     case PID_KD_ROT: if (len >= 4) can_kd_rot = unpack_float(d); break;
228
229     case MOTORS_ON:
230         wheel_pid_reset(&pid_left);
231         wheel_pid_reset(&pid_right);
232         can_ref_speed_left = can_ref_speed_right = 0.0f;
233         PWM_Enable(true);
234         can_motors_on = true;
235         break;
236
237     case MOTORS_OFF:
```

can_comm.c – process_msg() (nastavak)

```
238          can_motors_on = false;
239          can_ref_speed_left = can_ref_speed_right = 0.0f;
240          PWM_Enable(false);
241          wheel_pid_reset(&pid_left);
242          wheel_pid_reset(&pid_right);
243          break;
244
245      case FEEDBACK_ON:   can_feedback_on = true;   break;
246      case FEEDBACK_OFF:  can_feedback_on = false;  break;
247      case REFERENCE_ON:  can_reference_on = true;  break;
248      case REFERENCE_OFF:
249          can_reference_on = false;
250          can_ref_speed_left = can_ref_speed_right = 0.0f;
251          break;
252
253      case RESET_ENCODERS:
254          odometry_reset();
255          break;
256
257      case RESET_PID:
258          wheel_pid_reset(&pid_left);
259          wheel_pid_reset(&pid_right);
260          break;
261
262      default:
263          break;
264      }
265  }
```

100 — zadata brzina

kod	šta radi
`if (len < 8) return;`	Poruka mora imati 8 bajtova (dva float-a). Kraća poruka se odbacuje — bolje ignorisati nego čitati smeće iz memorije.
`last_cmd_tick = HAL_GetTick();` `cmd_timed_out = false;`	Zabeleži da je veza živa. **Ovo se radi i kad je referenca isključena** — poruka dokazuje da Raspberry radi, bez obzira da li joj se sadržaj koristi.
`if (!can_reference_on) return;`	Ako je referenca isključena (poruka 114), brzina se ignoriše.

`float wl, wr;`
`if (!unpack_float_checked(d, &wl) ||`
`!unpack_float_checked(d + 4, &wr))`
`return;`
	Raspakuj oba float-a uz proveru. Ako je **ijedan** NaN ili beskonačan, cela poruka se odbacuje — ne uzima se ni onaj ispravan, da točkovi ne bi dobili nepovezane komande.

`can_ref_speed_left  = clamp_speed(wl);` `can_ref_speed_right = clamp_speed(wr);`	Odseci na ±60 i upiši. **Bez konverzije** — rad/s ulazi, rad/s izlazi. Ovo su promenljive koje TIM7 prekid čita.

103 / 104 / 105 — pojačanja

kod	šta radi

`case CAN_ID_SET_KP:`
`{`
`float g;`
`if (len >= 4 &&`
`unpack_float_checked(d, &g))`
`wheel_gains.proportional_gain = g;`
`break;`
`}`
	Ista provera kao za brzinu. Upisuje se u **jedan zajednički skup** `wheel_gains` — oba regulatora ga vide istog trenutka. Ki i Kd potpuno isto.

109 / 110 — paljenje i gašenje motora

kod	šta radi

`case CAN_ID_MOTORS_ON:`
`  wheel_pid_reset(&pid_left);`
`  wheel_pid_reset(&pid_right);`
`  can_ref_speed_left = 0.0f;`
`  can_ref_speed_right = 0.0f;`
`  PWM_Enable(true);`
`  can_motors_on = true;`
	Redosled je važan. Prvo se čisti stanje regulatora i nuliraju reference, pa tek onda bude drajveri. Da je obrnuto, motor bi se probudio sa zatečenom starom komandom.

`case CAN_ID_MOTORS_OFF:`
`  can_motors_on = false;`
`  can_ref_speed_left = 0.0f;`
`  can_ref_speed_right = 0.0f;`
`  PWM_Enable(false);`
`  wheel_pid_reset(&pid_left);`
`  wheel_pid_reset(&pid_right);`
	Obrnuto: prvo se postavi zastavica (da regulaciona petlja odmah prestane da računa), pa se ugasi hardver, pa se očisti stanje. Regulator se resetuje da integrator ne bi „nakupio" grešku dok su motori ugašeni.

Ostale komande

poruka	šta radi
106–108	upisuje u `can_kp_rot` itd. — čuva se, ne koristi
111 / 112	menja `can_feedback_on`; dok je isključen, ništa se ne šalje nazad
113 / 114	menja `can_reference_on`; pri isključivanju odmah nulira reference
115	zove `odometry_reset()` — nulira brojače, zbirove i filter brzine
116	resetuje oba regulatora (stanje, ne koeficijente)

10.5 `HAL_FDCAN_RxFifo0Callback()` — prekid prijema

can_comm.c — HAL_FDCAN_RxFifo0Callback()

```
267  void HAL_FDCAN_RxFifo0Callback(FDCAN_HandleTypeDef *hfdcan, uint32_t RxFifo0ITs)
268  {
269      if ((RxFifo0ITs & FDCAN_IT_RX_FIFO0_NEW_MESSAGE) == 0)
270          return;
271
272      FDCAN_RxHeaderTypeDef rx;
273      uint8_t data[8];
274
275      while (HAL_FDCAN_GetRxFifoFillLevel(hfdcan, FDCAN_RX_FIFO0) > 0)
276      {
277          if (HAL_FDCAN_GetRxMessage(hfdcan, FDCAN_RX_FIFO0, &rx, data) != HAL_OK)
278              break;
279          if (rx.IdType != FDCAN_STANDARD_ID || rx.RxFrameType != FDCAN_DATA_FRAME)
280              continue;
281          process_msg(rx.Identifier, dlc_to_len(rx.DataLength), data);
282      }
283  }
```

kod	šta radi

`if ((RxFifo0ITs &`
`  FDCAN_IT_RX_FIFO0_NEW_MESSAGE) == 0)`
`  return;`
	Isti prekid može da se javi iz više razloga. Ovde se obrađuje samo „stigla nova poruka".

kod	šta radi

`while (HAL_FDCAN_GetRxFifoFillLevel(`
`hfdcan, FDCAN_RX_FIFO0) > 0)`
`{`
	Petlja, ne jedno čitanje. Dok je prekid bio zauzet moglo je stići više poruka; ako bi se čitala samo jedna, red bi se polako punio dok se ne prepuni.

`if (HAL_FDCAN_GetRxMessage(...) != HAL_OK)` `break;`	Ako čitanje ne uspe, izađi — bolje nego zaglaviti u beskonačnoj petlji unutar prekida.

`if (rx.IdType != FDCAN_STANDARD_ID ||`
`rx.RxFrameType != FDCAN_DATA_FRAME)`
`continue;`
	Odbaci proširene (29-bitne) ID-eve i remote frame-ove. Naš protokol koristi samo standardne podatkovne poruke.

`process_msg(rx.Identifier,`
`dlc_to_len(rx.DataLength), data);`
`}`
	Obradi poruku.

10.6 `can_comm_init()` i `can_comm_startup_flush()`

can_comm.c — can_comm_init()

```
287  void can_comm_init(void)
288  {
289      /* Prihvati ceo opseg protokola (100..152 dekadno) u RX FIFO0.
290       * Zahteva "Std Filters Nbr" >= 1 u CubeMX-u. */
291      FDCAN_FilterTypeDef f;
292      f.IdType       = FDCAN_STANDARD_ID;
293      f.FilterIndex  = 0;
294      f.FilterType   = FDCAN_FILTER_RANGE;
295      f.FilterConfig = FDCAN_FILTER_TO_RXFIFO0;
296      f.FilterID1    = CAN_ID_FILTER_MIN;   /* 100 */
297      f.FilterID2    = CAN_ID_FILTER_MAX;   /* 152 */
298      if (HAL_FDCAN_ConfigFilter(&hfdcan1, &f) != HAL_OK)
299          Error_Handler();
300
301      if (HAL_FDCAN_ConfigGlobalFilter(&hfdcan1, FDCAN_REJECT, FDCAN_REJECT,
302                                       FDCAN_REJECT_REMOTE, FDCAN_REJECT_REMOTE) != HAL_OK)
303          Error_Handler();
304
305      if (HAL_FDCAN_Start(&hfdcan1) != HAL_OK)
306          Error_Handler();
307
308      /* Prekidi za RX tek posle startup flush-a, da nule/smece sa starta ne
309       * budu obradjeni kao komande. Ukljucuje ih can_comm_startup_flush(). */
310      last_cmd_tick = HAL_GetTick();
311  }
```

can_comm.c — can_comm_startup_flush()

```
313 void can_comm_startup_flush(void)
314 {
315     /* 1) isprazni sopstveni RX FIFO neko vreme (kao u referentnom kodu) */
316     FDCAN_RxHeaderTypeDef rx;
317     uint8_t data[8];
318     uint32_t t0 = HAL_GetTick();
319     while (HAL_GetTick() - t0 < CAN_STARTUP_RX_DRAIN_MS)
320     {
321         while (HAL_FDCAN_GetRxFifoFillLevel(&hfdcan1, FDCAN_RX_FIFO0) > 0)
322             HAL_FDCAN_GetRxMessage(&hfdcan1, FDCAN_RX_FIFO0, &rx, data);
323         HAL_Delay(1);
324     }
325
326     /* 2) posalji par hiljada nula na feedback ID-jeve da se linux bafer ocisti.
327      * Ukupno trajanje je ograniceno - ako niko na busu ne daje ACK, TX FIFO
328      * ostaje pun i ne sme da nas zadrzi na startu. */
329     uint8_t zeros[8] = {0};
330     uint32_t t_flush = HAL_GetTick();
331     for (uint32_t i = 0; i < CAN_STARTUP_FLUSH_FRAMES; i++)
332     {
333         uint32_t id = (i & 1) ? WHEEL_POSITION : WHEEL_FEEDBACK_VEL;
334         while (!can_send(id, zeros, 8))
335         {
336             if (HAL_GetTick() - t_flush > CAN_STARTUP_FLUSH_MAX_MS)
337                 goto flush_done;
338         }
339     }
340 flush_done:
341
342     /* 3) sad tek pusti RX prekide */
343     if (HAL_FDCAN_ActivateNotification(&hfdcan1, FDCAN_IT_RX_FIFO0_NEW_MESSAGE, 0) != HAL_OK)
344         Error_Handler();
345
346     last_cmd_tick = HAL_GetTick();
347 }
```

kod	šta radi

`f.FilterType = FDCAN_FILTER_RANGE;`
`f.FilterConfig = FDCAN_FILTER_TO_RXFIFO0;`
`f.FilterID1 = CAN_ID_FILTER_MIN;`
`f.FilterID2 = CAN_ID_FILTER_MAX;`
	Prima ceo opseg protokola, **100 do 152**. Filter radi u hardveru — poruke van opsega se odbacuju pre nego što uopšte stignu do procesora, pa tuđi saobraćaj na busu ne pravi nijedan prekid. Granice su izvedene iz samog enum-a, pa se ne mogu razići sa njim.

`HAL_FDCAN_ConfigGlobalFilter(&hfdcan1,`
`  FDCAN_REJECT, FDCAN_REJECT,`
`  FDCAN_REJECT_REMOTE,`
`  FDCAN_REJECT_REMOTE);`
	Sve što ne uhvati filter — odbaci. Bez ovoga bi tuđe poruke na busu punile prijemni red.

`HAL_FDCAN_Start(&hfdcan1);`	Uključuje kontroler na magistralu. **Prekid se ovde još ne uključuje.**

`/* u can_comm_startup_flush() */`
`while (HAL_GetTick() - t0 <`
`  CAN_STARTUP_RX_DRAIN_MS)`
`{`
`  while (fill_level > 0)`
`  HAL_FDCAN_GetRxMessage(...);`
`  HAL_Delay(1);`
`}`
	Sekundu prazni sopstveni prijemni red — baca sve što je stiglo dok se ploča dizala.

kod	šta radi
`for (i = 0; i < CAN_STARTUP_FLUSH_FRAMES; i++) {   while (!can_send(id, zeros, 8))   if (HAL_GetTick() - t_flush >   CAN_STARTUP_FLUSH_MAX_MS)   goto flush_done; }`	Šalje 2000 nula-poruka da se isprazni bafer na Linux strani. Unutrašnja provera vremena je zaštita: ako niko ne sluša na busu, red za slanje ostane pun i ovo bi inače blokiralo start.
`HAL_FDCAN_ActivateNotification(&hfdcan1,   FDCAN_IT_RX_FIFO0_NEW_MESSAGE, 0);`	**Tek sada** se uključuje prekid prijema. Od ovog trenutka komande deluju.

10.7 Digitalni ulazi — `din_poll()`

can_comm.c — din_poll()

```
361  static void din_poll(din_t *d, uint32_t now)
362  {
363      uint8_t raw = (HAL_GPIO_ReadPin(d->port, d->pin) == GPIO_PIN_SET) ? 1 : 0;
364      if (!d->active_high)
365          raw ^= 1;
366
367      if (raw != d->raw_last) {
368          d->raw_last  = raw;          /* nije se jos smirilo - restartuj brojac */
369          d->raw_since = now;
370      } else if (d->stable != raw && (now - d->raw_since) >= d->debounce_ms) {
371          d->stable = raw;             /* stabilno dovoljno dugo - prihvati */
372          d->burst  = d->burst_count;
373      }
374
375      if (d->burst && can_send(d->id, (uint8_t *)&d->stable, 1))
376          d->burst--;
377  }
```

can_comm.c — inputs_check()

```
379  static void inputs_check(uint32_t now)
380  {
381      din_poll(&din_front, now);
382      din_poll(&din_back,  now);
383      din_poll(&din_cinch,  now);
384
385      can_cinch_state = (din_cinch.stable == 1) ? 1 : 0;
386
387  #if CAN_CINCH_HEARTBEAT_MS > 0
388      /* Ponovi stanje cinca povremeno, da ga cuje i ploca koja se kasnije upali */
389      if (din_cinch.stable != 0xFF && (now - cinch_hb_tick) >= CAN_CINCH_HEARTBEAT_MS) {
390          cinch_hb_tick = now;
391          if (din_cinch.burst == 0)
392              din_cinch.burst = 1;
393      }
394  #endif
395  }
```

Jedna funkcija opslužuje sva tri ulaza (prednji IR, zadnji IR, cinc). Razlikuju se samo po podacima u strukturi: koji pin, koji CAN ID, koji polaritet, koliko debounce.

kod	šta radi
`uint8_t raw = (HAL_GPIO_ReadPin(d->port, d->pin) == GPIO_PIN_SET) ? 1 : 0; if (!d->active_high) raw ^= 1;`	Pročitaj pin i, ako je senzor obrnutog polariteta, obrni vrednost. Ostatak funkcije ne mora da zna kakav je senzor — za nju je 1 uvek „aktivno".
`if (raw != d->raw_last) { d->raw_last = raw; d->raw_since = now; }`	**Prva grana: signal se upravo promenio.** Ne veruj mu još — samo zabeleži novu vrednost i trenutak, i kreni da meriš koliko će se zadržati.
`else if (d->stable != raw && (now - d->raw_since) >= d->debounce_ms) { d->stable = raw; d->burst = d->burst_count; }`	**Druga grana: signal se nije menjao dovoljno dugo.** Uslov ima dva dela: (1) nova vrednost se razlikuje od one koju smo do sada priznavali, i (2) prošlo je dovoljno vremena. Tek tada se promena priznaje i naredi se slanje.
`if (d->burst && can_send(d->id, &d->stable, 1)) d->burst--;`	Šalje **najviše jednu poruku po pozivu**. Da šalje svih 10 odjednom, zauzeo bi ceo red za slanje i zakasnio bi feedback. Brojač se smanjuje samo ako je slanje uspelo.

Ponavljanje stanja cinca

```
if (din_cinc.stable != 0xFF &&
    (now - cinc_hb_tick) >= CAN_CINC_HEARTBEAT_MS) {
    cinc_hb_tick = now;
    if (din_cinc.burst == 0)
        din_cinc.burst = 1;
}
```

Svake sekunde ponovi trenutno stanje cinca. Provera `burst == 0` sprečava da ponavljanje prekine burst koji je u toku. Vrednost `0xFF` znači „pin još nije očitan" — dok je tako, ništa se ne šalje.

10.8 `bus_off_recovery()`

```
FDCAN_ProtocolStatusTypeDef ps;
if (HAL_FDCAN_GetProtocolStatus(&hfdcan1, &ps) == HAL_OK && ps.BusOff)
    HAL_FDCAN_Start(&hfdcan1);
```

CAN kontroler ima ugrađenu zaštitu: ako broji previše grešaka (loš kabl, kratak spoj, pogrešan bitrate), sam se isključi sa magistrale — to je stanje **bus-off**. Iz njega **ne izlazi sam**. Bez ovih par redova, jedan loš kontakt bi zauvek ućutkao ploču i morao bi reset.

10.9 `can_comm_task()` — šta se vrti u glavnoj petlji

can_comm.c — can_comm_task()

```
407  void can_comm_task(void)
408  {
409      uint32_t now = HAL_GetTick();
410
411      bus_off_recovery();
412
413      if (feedback_due)
414      {
415          feedback_due = false;
416          if (can_feedback_on)
417          {
418              send_speed_feedback();
419              send_encoder_feedback();
420          }
421      }
422
423      inputs_check(now);
424
425  #if CAN_CMD_TIMEOUT_MS > 0
426      if (!cmd_timed_out && (now - last_cmd_tick) > CAN_CMD_TIMEOUT_MS)
427      {
428          cmd_timed_out = true;
429          can_ref_speed_left = can_ref_speed_right = 0.0f;
430      }
431  #endif
432  }
```

kod	šta radi
`uint32_t now = HAL_GetTick();`	Jedno očitavanje vremena za sve provere ispod.
`bus_off_recovery();`	Provera da li je kontroler ispao sa magistrale.
`if (feedback_due) { feedback_due = false; if (can_feedback_on) { send_speed_feedback(); send_encoder_feedback(); } }`	Zastavicu `feedback_due` postavlja TIM7 prekid svakih 10 ms. **Samo slanje se radi ovde, ne u prekidu** — prekid mora da bude kratak. Ako je feedback isključen, zastavica se svejedno briše (da se ne nakuplja).
`inputs_check(now);`	Obrada sva tri digitalna ulaza.
`if (CAN_CMD_TIMEOUT_MS > 0 && !cmd_timed_out && (now - last_cmd_tick) > CAN_CMD_TIMEOUT_MS) { cmd_timed_out = true; can_ref_speed_left = 0.0f; can_ref_speed_right = 0.0f; }`	Sigurnosni prekid. Zastavica `cmd_timed_out` osigurava da se nuliranje uradi **jednom**, a ne u svakom prolazu.

11. main.c — pokretanje i regulaciona petlja

11.1 Globalne promenljive

```
pid_gains_t wheel_gains;                  /* jedan skup Kp/Ki/Kd za oba tocka */
wheel_pid_t pid_left;
wheel_pid_t pid_right;

/* Menja ih TIM7 prekid, gledaju se iz debagera - volatile. */
volatile float meas_speed_left  = 0.0f;   /* rad/s, filtrirano */
volatile float meas_speed_right = 0.0f;
volatile float cmd_left  = 0.0f;          /* % maks. PWM-a, -100..+100 */
volatile float cmd_right = 0.0f;
```

Koeficijenti i dva regulatora su ovde, a ne u `can_comm.c`, jer ih koriste obe strane — regulaciona petlja ih pokreće, a CAN im menja pojačanja (zato ih `can_comm.c` uvozi sa `extern`).

Četiri `volatile` promenljive ne rade ništa u programu — postoje samo da bi mogao da ih dodaš u **Live Expressions** u debageru i uživo gledaš šta regulator radi.

11.2 `HAL_TIM_PeriodElapsedCallback()` — regulaciona petlja, 500 Hz

Ovo je funkcija koju HAL automatski zove kad TIM7 istekne. Sve unutar nje se izvršava svake 2 ms.

main.c — HAL_TIM_PeriodElapsedCallback()

```
 89  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
 90  {
 91      if (htim->Instance != TIM7)
 92          return;
 93
 94      static uint16_t feedback_div = 0;
 95      static uint16_t alive_div = 0;
 96
 97      odometry_update();                      /* razlika brojaca -> rad/s (+ filter) */
 98      meas_speed_left  = get_wheel_speed_left();
 99      meas_speed_right = get_wheel_speed_right();
100
101      if (can_motors_on)
102      {
103          cmd_left  = wheel_pid_update(&pid_left,  can_ref_speed_left,  meas_speed_left);
104          cmd_right = wheel_pid_update(&pid_right, can_ref_speed_right, meas_speed_right);
105          PWM_SetCommand_Left(cmd_left);
106          PWM_SetCommand_Right(cmd_right);
107      }
108      else
109      {
110          cmd_left = cmd_right = 0.0f;
111          PWM_Stop();
112      }
113
114      if (++feedback_div >= (CONTROL_FREQ_HZ / FEEDBACK_FREQ_HZ))
115      {
116          feedback_div = 0;
117          can_comm_feedback_due();
118      }
119
120      if (++alive_div >= (CONTROL_FREQ_HZ / 2))   /* 1 Hz treptanje = petlja ziva */
121      {
122          alive_div = 0;
123          HAL_GPIO_TogglePin(ALIVE_GPIO_Port, ALIVE_Pin);
124      }
125  }
```

kod	šta radi
`if (htim->Instance != TIM7) return;`	Isti callback bi se zvao i za druge tajmere. Ovde nas zanima samo TIM7 — za sve ostalo izlazimo odmah.
`static uint16_t feedback_div = 0;` `static uint16_t alive_div = 0;`	`static` znači da zadrže vrednost između poziva. To su delioci frekvencije: od 500 Hz prave 100 Hz i 1 Hz.
`odometry_update();`	Očitaj enkodere, izračunaj rad/s, provuci kroz filter. **Mora prvo** — sve ostalo zavisi od ovoga.
`meas_speed_left  = get_wheel_speed_left();` `meas_speed_right = get_wheel_speed_right();`	Uzmi filtrirane brzine.
`if (can_motors_on) { cmd_left = wheel_pid_update(&pid_left, can_ref_speed_left, meas_speed_left); cmd_right = wheel_pid_update(&pid_right, can_ref_speed_right, meas_speed_right); PWM_SetCommand_Left(cmd_left); PWM_SetCommand_Right(cmd_right); }`	**Srž petlje.** Za svaki točak: iz zadate i izmerene brzine (rad/s) izračunaj komandu u %, pa je pošalji na PWM. Dva regulatora su nezavisna po stanju (svaki ima svoj integrator), ali dele koeficijente.

kod	šta radi
`else { cmd_left = cmd_right = 0.0f; PWM_Stop(); }`	Kad su motori ugašeni preko CAN-a, regulator se **ne poziva uopšte**. Da se poziva, integrator bi nakupljao grešku (jer se robot ne kreće) i pri paljenju bi motor naglo poskočio.
`if (++feedback_div >= (CONTROL_FREQ_HZ / FEEDBACK_FREQ_HZ)) { feedback_div = 0; can_comm_feedback_due(); }`	500/100 = svaki peti prolaz podigne zastavicu. **Poruka se ovde ne šalje** — samo se obeleži da treba; slanje radi glavna petlja. Nema više „latch" prosečne brzine — filtrirana brzina je uvek spremna.
`if (++alive_div >= (CONTROL_FREQ_HZ / 2)) { alive_div = 0; HAL_GPIO_TogglePin(ALIVE_GPIO_Port, ALIVE_Pin); }`	Svakih 250 prolaza (pola sekunde) promeni stanje ALIVE pina → LED trepće na **1 Hz**. Ako LED ne trepće ili trepće pogrešnom brzinom, odmah znaš da regulaciona petlja ne radi ili radi na pogrešnoj frekvenciji.

U prekidu **nema nijednog slanja poruke, nema čekanja, nema** `HAL_Delay`. Sve što traje neodređeno dugo je prebačeno u glavnu petlju. Prekid mora da se završi pre nego što stigne sledeći — na 500 Hz to je 2 ms, a ovaj posao traje nekoliko desetina mikrosekundi.

11.3 `main()` — deo koji smo dodali

Sve do `/* USER CODE BEGIN 2 */` je CubeMX kod i ne dira se. Evo šta je dodato posle:

kod	šta radi
`__HAL_TIM_SET_PRESCALER(&htim7, TIM7_PRESCALER); __HAL_TIM_SET_AUTORELOAD(&htim7, (TIM7_INPUT_CLK_HZ / (TIM7_PRESCALER + 1)) / CONTROL_FREQ_HZ - 1);`	Namešta TIM7 na tačno 500 Hz **iz koda**, bez obzira šta piše u CubeMX-u. Računica: 64 MHz / 64 = 1 MHz brojački takt; 1 MHz / 500 = 2000 tikova po periodi; ARR = 1999.
`HAL_TIM_GenerateEvent(&htim7, TIM_EVENTSOURCE_UPDATE); __HAL_TIM_CLEAR_FLAG(&htim7, TIM_FLAG_UPDATE);`	Registri PSC i ARR su baferovani — nove vrednosti stupaju na snagu tek posle „update" događaja. Prvi red ga izaziva ručno, a drugi briše zastavicu koju je time podigao, da prekid ne opali odmah po pokretanju.
`wheel_pid_gains_init(&wheel_gains, PID_DEFAULT_KP, PID_DEFAULT_KI, PID_DEFAULT_KD, PID_D_LPF_FC_HZ, (float)CONTROL_FREQ_HZ);`	Jedan skup koeficijenata + koeficijent filtera D člana. **Pre** uključenja CAN prekida — da poruka koja bi stigla u toj sekundi ne bude prebrisana.
`wheel_pid_init(&pid_left, &wheel_gains, PID_COMMAND_LIMIT_PCT, PID_INTEGRATOR_LIMIT_PCT); wheel_pid_init(&pid_right, &wheel_gains, ...);`	Oba regulatora pokazuju na **isti** skup.
`PWM_Init(); encoder_init(); odometry_init();`	Plafon ispune iz napona + budi drajvere; startuje brojače; računa koeficijent filtera brzine i nulira zbirove. Nema više prečnika ni razmaka točkova.
`can_comm_init(); can_comm_startup_flush();`	Filter i start CAN-a, pa pražnjenje bafera. Traje oko 1–3 sekunde — **to je namerno** i dešava se pre nego što motori mogu da se pokrenu.

kod	šta radi
`HAL_TIM_Base_Start_IT(&htim7);`	**Poslednji red pre petlje.** Od ovog trenutka regulator radi. Namerno je ovde, da se ne pokrene dok enkoderi i motori nisu spremni.

`while (1)`
`{`
`  can_comm_task();`
`}`
	Cela glavna petlja. Vrti se što brže može; sve ostalo se dešava kroz prekide.

12. Brzi podsetnik

12.1 Ključni brojevi

veličina	vrednost	odakle
frekvencija regulacije	500 Hz (svake 2 ms)	`CONTROL_FREQ_HZ`
frekvencija feedback-a	100 Hz (svakih 10 ms)	`FEEDBACK_FREQ_HZ`
CAN brzina	500 kbit/s	CubeMX, FDCAN1
takt FDCAN-a	40 MHz (PLL2Q)	CubeMX, Clock Configuration
rezolucija enkodera	8192 tika po obrtaju	2048 CPR × 4 (mod TI12)
ugao po jednom tiku	0.000767 rad	`RAD_PER_TICK`
kvantizacija brzine	0.38 rad/s po tiku u prolazu	RAD_PER_TICK × 500
prelivanje desnog brojača	svakih 8 obrtaja	65536 tikova (TIM3 je 16-bitni)
najveća ispuna	60 % = CCR 1536	18 V / 30 V
najveća brzina	**~29.4 rad/s**	fizički, pri 18 V
odsecanje reference	±60 rad/s	`MAX_WHEEL_SPEED_RADS`
filter brzine	30 Hz, a = 0.686	`SPEED_LPF_FC_HZ`
filter D člana	20 Hz, a = 0.778	`PID_D_LPF_FC_HZ`
početna pojačanja	Kp 5.7, Ki 0.118, Kd 0	`PID_DEFAULT_*`
opterećenje magistrale	~8.5 % pri svemu uključenom	izmereno simulacijom

12.2 Redosled koji ros2 treba da pošalje pri startu

```
109  MOTORS_ON          probudi drajvere, očisti regulatore
103  Kp                  (opciono — ako ne šalješ, ostaje 5.7)
104  Ki                  (opciono — ako ne šalješ, ostaje 0.118)
105  Kd                  (opciono — ako ne šalješ, ostaje 0; šalji 0 da isključiš D)
115  RESET_ENCODERS      nuliraj brojače pre početka
111  FEEDBACK_ON         od sada STM šalje 101 i 102

zatim, 100 puta u sekundi:
100  [leva rad/s][desna rad/s]
```

Feedback je pri paljenju isključen. To je namerno — dok strategija ne pozove `init()`, STM ćuti i ne puni bafer na Linux strani. Ako zaboraviš da pošalješ 111, nikad nećeš dobiti ni brzine ni enkodere.

12.3 Provera da ploča radi (bez opreme)

znak	šta znači
ALIVE LED trepće na 1 Hz	regulaciona petlja radi na tačno 500 Hz
LED ne trepće	TIM7 ne radi — proveri da li je `HAL_TIM_Base_Start_IT` pozvan i NVIC uključen
LED trepće pogrešnom brzinom	`TIM7_INPUT_CLK_HZ` ne odgovara stvarnom taktu
stižu 150/151/152 na start	CAN slanje radi, senzori se čitaju
ne stiže 101 iako vozi	nije poslat 111 FEEDBACK_ON
brojevi u 101 su besmisleni	najverovatnije **redosled bajtova** — probaj `CAN_LITTLE_ENDIAN 1`
zadaš 10 rad/s, točak ide unazad	obrnut enkoder ili DIR — vidi 12.4

12.4 Šta još nije potvrđeno na pravom robotu

stavka	gde se menja	kako se proverava
rezolucija enkodera 2048 CPR	`ENCODER_TICKS_PER_REV`	okreni točak tačno jedan pun krug rukom i vidi da li brojač u 102 poraste za 8192
smer brojanja enkodera	`ENC_LEFT_SIGN` / `ENC_RIGHT_SIGN`	guraj robota napred rukom: oba zbira u 102 moraju da **rastu**; ako neki opada, stavi −1
smer motora	`PWM_SetCommand_*` u `motors.c`	zadaj +5 rad/s samo jednom točku: mora da se okreće napred; ako ne, zameni 0/1 u SetDir
polaritet cinca	`CINC_ACTIVE_HIGH`	izvuci cinc i vidi da li 152 pošalje 1
polaritet IR senzora	`IR_ACTIVE_HIGH`	stavi ruku ispred senzora i vidi da li 150 pošalje 1
redosled bajtova	`CAN_LITTLE_ENDIAN`	pošalji poznatu brzinu i vidi da li se u 101 vrati približno ista
ID-jevi na ros2 strani	`enum CanID` u `can_comm.h`	moraju biti dekadni 100–152, identični ros2 enum-u; ako se ne poklope, ništa ne stiže
napon napajanja	`MOTOR_SUPPLY_VOLTAGE_DEFAULT`	izmeri na drajveru pri punoj bateriji; ako nije 30 V, promeni broj (dok ne dođe ADC)

Prve dve stavke su najvažnije. **Ako je smer enkodera pogrešan, regulator radi naopako** — vidi brzinu sa pogrešnim znakom, „ispravlja" je u pogrešnu stranu i motor odleti u zasićenje. To se prepozna odmah: na najmanju komandu točak ode na punu brzinu. Prvo proveri smer guranjem rukom, tek onda pali motore.

12.5 Šta se promenilo u odnosu na prethodnu verziju

ranije	sada	zašto
sve u mm/s, konverzija na granici CAN-a	sve u rad/s, bez konverzije	nema greške kalibracije prečnika; STM-u ne trebaju dimenzije
inkrementalni PI (bez D)	pozicioni PID sa anti-windup-om i filtriranim D	tvoja PIC32 matematika; D postoji da se ne bi dopisivao mesec dana pred takmičenje
po jedan skup pojačanja po točku	jedan zajednički skup	nemoguće je da se točkovi razlikuju
regulator gledao sirovu brzinu, CAN dobijao prosek	oba dobijaju istu filtriranu brzinu	manje šuma u petlji, jedan mehanizam
napon napajanja kao #define	globalna promenljiva + `PWM_SetSupplyVoltage()`	spremno za ADC
x, y, ugao računani na STM-u	isključeno (`ODOMETRY_ON_STM` = 0)	radi ih ros2
float sa CAN-a se upisivao direktno	provera NaN/Inf + odsecanje reference	NaN bi zaglavio integrator i mogao da da 100 % PWM-a
kratka imena u strukturi (kp, err, u)	puna imena (proportional_gain, error, command)	da drugi mogu da čitaju
ID-jevi 0x100–0x152 (heks)	ID-jevi 100–152 (dekadno), imena iz ros2 enum-a CanID	**heks 0x100 je broj 256** — strane se ne bi čule; sada su prepisani doslovno

12.6 Kako je kod proveren

Ceo sistem — ros2 koji šalje komande, CAN magistrala sa stvarnim trajanjem poruka, firmver preveden red po red u simulaciju, model motora i kvantizacija enkodera — vrti se u simulaciji pre nego što je išta flešovano.

šta	rezultat
90 ciljanih testova (svaka CAN komanda, vožnja, zasićenje, timeout, reseti, prelivanje brojača, IR, cinc, filter, D član, napon, NaN, smer enkodera, poklapanje CAN ID-jeva)	**90 od 90 prolazi**
75 nasumičnih scenarija (motor, napon baterije 21–30 V, pojačanja, reference do ±35 rad/s, nasumični početni brojači, paljenje/gašenje usred vožnje)	napon na motoru **nikad preko 18.00 V**, CCR nikad preko plafona, zbir enkodera tačan u 2 tika, nigde NaN
poređenje sa starim regulatorom na istom motoru	isti odziv: t90 160 ms vs 166 ms, preskok 0.06 % vs 0.11 %
kompajliranje (arm-none-eabi-gcc iz CubeIDE-a, -Wall -Wextra)	nula upozorenja, u obe varijante `ODOMETRY_ON_STM`

Dokument opisuje stanje koda posle prelaska na rad/s i pozicioni PID. Sve tvrdnje o ponašanju (šum u feedback-u, vreme odziva, opterećenje magistrale, ponašanje pri prelivanju brojača, oporavak od zasićenja) su izmerene u simulaciji celog sistema.